Programming with Python

Course code 202044504

Assignment 2 — Solutions

B.Tech. Information Technology · Semester V

Student name	MOXA PATEL
Enrollment no.	12402080601085
Batch	2A8
Date of submission	

10 programs · Python 3.10+ · MySQL · Tkinter · matplotlib · 79 of 79 tests passed

Contents

Q1. Regex-Based Email Intelligence Extractor 3

Q2. Offline Web Scraping Ranker from HTML Pages 6

Q3. MySQL Student-Course Analytics Engine 12

Q4. SQL Join Query Builder and Validator 18

Q5. Tkinter Contact Manager with MySQL Backend 27

Q6. Log Anomaly Detector using Sliding Window 35

Q7. Generator-Based JSON ETL Pipeline 39

Q8. Matrix Path Optimizer with Dynamic Programming 44

Q9. OOP Inventory Merger with Operator Overloading 48

Q10. Python Data Visualization Dashboard Exporter 54

How the answers are organised

- Each question has: the problem in brief, the approach and data structures, assumptions where the paper is silent, time and space complexity, how to run it, the complete source code, and the test evidence.
- Every program validates its input, uses meaningful function names and comments, and was run on the sample test, boundary tests, invalid-input tests, at least three self-made tests and a large test at the maximum constraints.
- Source files follow the convention `Enrollment_AssignmentNo_QNo.py` with `ENROLLMENT` as a placeholder for the enrollment number.
- Environment: Python 3.12 on Linux. Extra packages, only where the question needs them: `mysql-connector-python` (Q3, Q4, Q5) and `matplotlib` (Q10); Tkinter for Q5. Database tests ran on a real MariaDB 10.11 server (MySQL-compatible). The same files run unchanged on Windows and macOS with Python 3.10 or newer.

Test summary

Question	Tests	Passed	Largest input tested
Q1. Regex-Based Email Intelligence Extractor	6	6	120 MB file, 1.5 M emails
Q2. Offline Web Scraping Ranker from HTML Pages	6	6	40 pages, 400,000 products
Q3. MySQL Student-Course Analytics Engine	9	9	1 M students + 5 M registrations
Q4. SQL Join Query Builder and Validator	14	14	8 rejection cases, injection test
Q5. Tkinter Contact Manager with MySQL Backend	7	7	10,000 contacts
Q6. Log Anomaly Detector using Sliding Window	8	8	n = 2,000,000 log lines
Q7. Generator-Based JSON ETL Pipeline	5	5	3,000,000 records (259 MB)
Q8. Matrix Path Optimizer with Dynamic Programming	11	11	2000 × 2000 matrix
Q9. OOP Inventory Merger with Operator Overloading	5	5	2 × 500,000 products, 300,000 ops
Q10. Python Data Visualization Dashboard Exporter	8	8	1,000,000 students × 10 subjects

Q1. Regex-Based Email Intelligence Extractor

CO-1, CO-3, CO-4 | Bloom L4 Analyze | Regular expressions, text filtering, sets, dictionaries, file streaming

Problem

From a large noisy text file, extract every valid email whose top-level domain is com, edu or org (user part: letters, digits, + . _ -). For each domain print the number of unique users and the lexicographically smallest email; ignore invalid emails.

Approach

- One compiled regular expression finds the emails. The user part is `[A-Za-z0-9+._-]+`; the domain is one or more labels (letters, digits, hyphens, not starting or ending with a hyphen) followed by `.com`, `.edu` or `.org`.
- **Boundary checks** make the match all-or-nothing. A negative lookbehind stops a match from starting in the middle of other email characters, and a negative lookahead stops `x@site.com.au` from being read as `x@site.com`. As a result `root@@bad.com`, `a@b@c.com` and `x@-site.com` are rejected entirely (all covered by tests).
- The file is read **line by line**, and a generator yields (user, domain) pairs, so a 1 GB file never sits in memory; only the unique emails do.
- A dictionary maps domain to a **set** of user parts, so duplicates are counted once. A second dictionary keeps the smallest email seen per domain, updated in O(1) per email instead of sorting at the end.
- Domains are lower-cased because they are case-insensitive (`Adit.EDU` and `adit.edu` are the same domain); the user part is kept exactly as written.

Complexity

Time O(total characters): the regex scans each line once. **Space** O(unique valid emails) for the per-domain sets.

How to run

```
python ENROLLMENT_2_Q1.py emails.txt
```

Source code — ENROLLMENT_2_Q1.py

```python
"""
Q1. Regex-Based Email Intelligence Extractor
Course : Programming with Python (202044504) - Assignment 2
CO     : CO-1, CO-3, CO-4

Reads a (possibly very large) text file line by line, extracts every valid
email whose top-level domain is com, edu or org, and prints for each domain
(in lexicographic order):  domain  unique_user_count  smallest_email

Valid email (as defined for this program)
    user   : 1+ of  letters digits + . _ -
    domain : one or more labels separated by dots, each label made of
             letters/digits/hyphens, not starting or ending with a hyphen,
             and ending in .com / .edu / .org
    The match must not touch other email characters on either side, so
    "root@@bad.com", "a@b@c.com" and "x@site.com.au" are rejected as a whole.

Normalisation: the domain is lower-cased (domains are case-insensitive);
the user part is kept as written. Duplicate emails are counted once.

Run   : python ENROLLMENT_2_Q1.py emails.txt
Time  : O(total characters)          Space : O(unique valid emails)
"""

import re
import sys
from collections import defaultdict

LABEL = r"[A-Za-z0-9](?:[A-Za-z0-9-]{0,61}[A-Za-z0-9])?"
EMAIL_RE = re.compile(
    r"(?<![A-Za-z0-9+._@-])"                  # not glued to another email char
    r"([A-Za-z0-9+._-]+)"                      # user part
    r"@"
    rf"((?:{LABEL}\.)+(?:com|edu|org))"        # domain ending in an allowed TLD
    r"(?![A-Za-z0-9_@-]|\.[A-Za-z0-9])",       # TLD must really be the end
    re.IGNORECASE)


def extract_emails(lines):
    """Generator: yields (user, domain) for every valid email in the lines."""
    for line in lines:
        for match in EMAIL_RE.finditer(line):
            user, domain = match.group(1), match.group(2).lower()
            yield user, domain


def summarise(lines):
    """Return {domain: (unique_user_count, smallest_email)}."""
    users = defaultdict(set)            # domain -> set of user parts (dedupe)
    smallest = {}                       # domain -> lexicographically smallest email
    for user, domain in extract_emails(lines):
        users[domain].add(user)
        email = f"{user}@{domain}"
        if domain not in smallest or email < smallest[domain]:
            smallest[domain] = email
    return {d: (len(users[d]), smallest[d]) for d in sorted(users)}


def main():
    if len(sys.argv) != 2:
        print("Usage: python ENROLLMENT_2_Q1.py <text_file>")
        sys.exit(1)
    path = sys.argv[1]
    try:
        # errors="replace": a stray binary byte must not stop a 1 GB scan
        with open(path, encoding="utf-8", errors="replace") as fh:
            result = summarise(fh)
    except FileNotFoundError:
        print(f"ERROR: file not found: {path}")
        sys.exit(1)
    except (IsADirectoryError, PermissionError) as err:
        print(f"ERROR: cannot read {path}: {err}")
```

```
            sys.exit(1)

        if not result:
            print("NO_VALID_EMAILS")
        for domain, (count, email) in result.items():
            print(domain, count, email)


if __name__ == "__main__":
    main()
```

Test cases — 6 of 6 passed

#	Test case	Input	Output (expected = actual)	Result
1	**Sample test** sample	emails.txt: Contact a.b+1@adit.edu, root@@bad.com, test-user@x.org and a.b+1@adit.edu	adit.edu 1 a.b+1@adit.edu x.org 1 test-user@x.org	**PASS**
2	**Case-insensitive domain, punctuation, 6 malformed/other-TLD emails** custom	tricky.txt: Mail Ravi@CS.Adit.EDU. or ravi@cs.adit .edu; Ravi@cs.adit.edu! bad: x@-site.com y@site-.org a@b@c.com z@site.com.au q@site.net w@x.co.in info_desk@shop.com sales-team+i n@shop.com (zzz@shop.com)	cs.adit.edu 2 Ravi@cs.adit.edu shop.com 3 info_desk@shop.com	**PASS**
3	**Boundary: 1-character users, duplicate after domain lower-casing** boundary	dots.txt: first.last@mail.org,first.last@MAIL.OR G _@x.com -@x.com	mail.org 1 first.last@mail.org x.com 2 -@x.com	**PASS**
4	**Boundary: no valid email** boundary	empty.txt: no emails here, only @ signs @@ and do ts...	NO_VALID_EMAILS	**PASS**
5	**Invalid: file does not exist** invalid	missing.txt	ERROR: file not found: missing.txt	**PASS**
6	**Large: 120 MB file, 1.5 million emails** performance	500,000 lines, 3 valid + 1 invalid ema il each	finished in 5.22 s	**PASS**

Q2. Offline Web Scraping Ranker from HTML Pages

CO-1, CO-3, CO-4 | Bloom L5 Evaluate | Web scraping, HTML parsing, sorting, data cleaning, dictionaries

Problem

From saved product-listing HTML pages, extract name, price and rating; normalise prices, remove duplicate products by name, and print the top K by rating (ties: lower price, then name).

Approach

- The standard-library `html.parser` is used instead of regex on raw HTML, because nested tags (`<h2>Desk <b>Lamp</b></h2>`) break simple patterns. The parser tracks nesting depth: a container with class `product`, `product-card` or `product-item` opens a record, and child elements whose class contains name/title, price or rating supply its fields.
- Regular expressions do the **cleaning**. Currency markers (Rs, INR, $, the rupee sign) and thousands separators are removed and the price is parsed as a Decimal, which must be above 0. The rating is the first number in the text, so `4.8 out of 5` and `4.7/5` both work, and it must lie between 0 and 5. HTML entities such as `&` are decoded.
- **Duplicates:** names are compared case-insensitively in a dictionary, and the better offer (higher rating, then lower price) is kept. A block with a missing or invalid field is skipped and counted.
- Each file is fed to the parser in 1 MB chunks, so even very large pages stream through. Only one entry per unique product is kept, and those u products are sorted once.
- Sample pages `page1.html` and `page2.html` are included in the Q2 folder; they reproduce the sample output exactly.

Complexity

Time O(total HTML size + u log u): one pass over the HTML, then one sort of the u unique products. **Space** O(u).

The paper does not give the HTML layout of the pages, so the program defines one (shown in the source docstring) and accepts the common variants of it: different tag names, extra classes, and nested markup.

How to run

```
python ENROLLMENT_2_Q2.py < sample_input.txt
```

Source code — ENROLLMENT_2_Q2.py

```python
"""
Q2. Offline Web Scraping Ranker from HTML Pages
Course : Programming with Python (202044504) - Assignment 2
CO     : CO-1, CO-3, CO-4

Reads saved product-listing HTML pages, extracts (name, price, rating) for
every product, cleans the values, removes duplicate products by name and
prints the top K by rating (ties: lower price, then product name).

Expected page structure (any tag names, matched by CSS class):
    <div class="product">                       container: class product / product-card / product-item
        <h2 class="name">Python Book</h2>         class contains "name" or "title"
        <span class="price">Rs. 1,299.00</span>   class contains "price"
        <span class="rating">4.8 / 5</span>       class contains "rating"
    </div>

Cleaning
    price  : currency words/symbols (Rs, INR, $, rupee sign) and commas
             removed -> Decimal; must be > 0
    rating : first number in the text (handles "4.8", "4.8/5", "4.8 out of 5")
             must be between 0 and 5
    name   : HTML entities decoded, extra spaces collapsed
    Products with a missing or invalid field are skipped and counted.
Duplicates: names compared case-insensitively; the better offer is kept
            (higher rating, then lower price).

html.parser (standard library) is used instead of regex on raw HTML because
regex cannot follow nested tags reliably. Each file is fed to the parser in
1 MB chunks, so a large page is never loaded into memory at once.

Run    : python ENROLLMENT_2_Q2.py < input.txt
         input.txt: first line "p k", then p HTML file paths
Time   : O(total HTML size + u log u)        Space : O(u) unique products
"""

import re
import sys
from decimal import Decimal, InvalidOperation
from html.parser import HTMLParser

CHUNK = 1 << 20
CONTAINER_CLASSES = {"product", "product-card", "product-item"}
VOID_TAGS = {"area", "base", "br", "col", "embed", "hr", "img", "input", "link",
             "meta", "param", "source", "track", "wbr"}
NUMBER_RE = re.compile(r"\d+(?:\.\d+)?")
PRICE_CLEAN_RE = re.compile(r"(?i)rs\.?|inr|[$₹,\s]")


class InputError(Exception):
    pass


def field_of(classes):
    """Map an element's class list to the field it holds, if any."""
    for cls in classes:
        c = cls.lower()
        if "price" in c:
            return "price"
        if "rating" in c:
            return "rating"
        if "name" in c or "title" in c:
            return "name"
    return None


class ProductParser(HTMLParser):
    """Collects raw (name, price, rating) text for every product container."""

    def __init__(self):
        super().__init__(convert_charrefs=True)
        self.depth = 0
        self.product_depth = None       # depth of the open product container
```

```
        self.field = None               # field currently being read
        self.field_depth = None
        self.current = {}
        self.raw_products = []

    def handle_starttag(self, tag, attrs):
        if tag in VOID_TAGS:
            return
        self.depth += 1
        classes = (dict(attrs).get("class") or "").split()
        if self.product_depth is None:
            if any(c.lower() in CONTAINER_CLASSES for c in classes):
                self.product_depth = self.depth
                self.current = {}
            return
        if self.field is None:
            field = field_of(classes)
            if field and field not in self.current:
                self.field, self.field_depth = field, self.depth
                self.current[field] = ""

    def handle_endtag(self, tag):
        if tag in VOID_TAGS:
            return
        if self.field is not None and self.depth == self.field_depth:
            self.field = None
        if self.product_depth is not None and self.depth == self.product_depth:
            self.raw_products.append(self.current)
            self.product_depth = None
        self.depth = max(0, self.depth - 1)

    def handle_data(self, data):
        if self.field is not None:
            self.current[self.field] += data


def clean_price(text):
    try:
        value = Decimal(PRICE_CLEAN_RE.sub("", text))
    except InvalidOperation:
        return None
    return value if value.is_finite() and value > 0 else None


def clean_rating(text):
    m = NUMBER_RE.search(text)
    if not m:
        return None
    value = Decimal(m.group())
    return value if 0 <= value <= 5 else None


def clean(raw):
    """Return (name, price, rating) or None if any field is missing/invalid."""
    name = " ".join(raw.get("name", "").split())
    price = clean_price(raw.get("price", ""))
    rating = clean_rating(raw.get("rating", ""))
    if not name or price is None or rating is None:
        return None
    return name, price, rating


def rank_key(product):
    name, price, rating = product
    return (-rating, price, name)


def parse_file(path):
    parser = ProductParser()
    with open(path, encoding="utf-8", errors="replace") as fh:
        while chunk := fh.read(CHUNK):
            parser.feed(chunk)
```

```
    parser.close()
    return parser.raw_products


def rank(paths, k, log=sys.stderr):
    best = {}                           # lower-case name -> product tuple
    skipped = 0
    for path in paths:
        try:
            raws = parse_file(path)
        except OSError as err:
            print(f"WARNING: skipped {path}: {err.strerror}", file=log)
            continue
        for raw in raws:
            product = clean(raw)
            if product is None:
                skipped += 1
                continue
            key = product[0].lower()
            if key not in best or rank_key(product) < rank_key(best[key]):
                best[key] = product
    if skipped:
        print(f"INFO: {skipped} product block(s) skipped (missing or invalid  fields)", file=log)
    return sorted(best.values(), key=rank_key)[:k]


def fmt(value):
    """Decimal -> '499' or '499.5' (no trailing zeros)."""
    text = format(value.normalize(), "f")
    return text


def solve(text, log=sys.stderr):
    lines = [ln.strip() for ln in text.splitlines() if ln.strip()]
    if not lines:
        raise InputError("empty input")
    try:
        p, k = map(int, lines[0].split())
    except ValueError:
        raise InputError("first line must be: p k") from None
    if not (1 <= p <= 5000 and 1 <= k <= 1000):
        raise InputError("p or k outside constraints")
    if len(lines) - 1 < p:
        raise InputError(f"expected {p} file paths")
    top = rank(lines[1:p + 1], k, log)
    if not top:
        return "NO_PRODUCTS"
    return "\n".join(f"{name} {fmt(price)} {fmt(rating)}" for name,  price, rating in top)


def main():
    try:
        print(solve(sys.stdin.read()))
    except InputError as err:
        print(f"INPUT ERROR: {err}")
        sys.exit(1)


if __name__ == "__main__":
    main()
```

Test cases — 6 of 6 passed

#	Test case	Input	Output (expected = actual)	Result
1	**Sample test (page1.html, page2.html)** sample	2 2 page1.html page2.html	Python Book 499 4.8 USB Keyboard 799 4.7	**PASS**
2	**All products, duplicates removed, invalid block skipped** custom	2 10 page1.html page2.html	Python Book 499 4.8 USB Keyboard 799 4.7 Wireless Mouse 1049 4.5 Pen & Paper Set 12.5 4.2 [stderr] INFO: 1 product block(s) skipped (miss ing or invalid fields)	**PASS**

#	Test case	Input	Output (expected = actual)	Result

3	**Nested tags in name, price 0 / 'free' / rating 6.0 rejected, tie-breaks**
	custom
		`1 5`
		`odd.html (7 products: Desk <b>Lamp</b> 350 4.9, Desk Mat price 0,`
		`Desk Fan 'free', Chair rating 6.0, Table/Bench 2500 4.0, Stool 999.5 4)`
			`Desk Lamp 350 4.9`
			`Stool 999.5 4`
			`Bench 2500 4`
			`Table 2500 4`
			`[stderr]`
			`INFO: 3 product block(s) skipped (missing or invalid fields)`
				PASS

4	**Boundary: k=1, one missing file is skipped**
	boundary
		`2 1`
		`nope.html`
		`page2.html`
			`USB Keyboard 899 4.7`
			`[stderr]`
			`WARNING: skipped nope.html: No such file or directory`
			`INFO: 1 product block(s) skipped (missing or invalid fields)`
				PASS

5	**Invalid: header not two integers** invalid	`two 2` `page1.html`	`INPUT ERROR: first line must be: p k`	**PASS**
6	**Large: 40 pages, 400,000 products, ~120 MB** performance	`random products with duplicates`	`finished in 21.44 s`	**PASS**

Sample file — page1.html

```
<!doctype html>
<html><head><meta charset="utf-8"><title>Shop · page 1</title></head>
<body>
<div class="product-list">
  <div class="product">
    <h2 class="name">Python Book</h2>
    <span class="price">Rs. 499</span>
    <span class="rating">4.8 out of 5</span>
  </div>
  <div class="product card">
    <h2 class="product-name">USB  Keyboard</h2>
    <img src="kb.png" alt="keyboard">
    <span class="price">&#8377;799.00</span>
    <span class="rating">4.7/5</span>
  </div>
  <div class="product">
    <h2 class="name">Wireless Mouse</h2>
    <span class="price">INR 1,049</span>
    <span class="rating">4.5</span>
  </div>
</div>
</body></html>
```

Sample file — p a g e 2 . h t m l

```
<!doctype html>
<html><body>
<section>
  <article class="product-item">
    <h3 class="title">python book</h3>
    <p class="price">Rs. 549</p>
    <p class="rating">4.6</p>
  </article>
  <article class="product-item">
    <h3 class="title">USB Keyboard</h3>
    <p class="price">Rs. 899</p>
    <p class="rating">4.7</p>
  </article>
  <article class="product-item">
    <h3 class="title">Laptop Stand</h3>
    <p class="price">Rs. 1,299</p>
  </article>
  <article class="product-item">
    <h3 class="title">Pen &amp;amp; Paper Set</h3>
    <p class="price">$12.50</p>
    <p class="rating">4.2</p>
  </article>
</section>
</body></html>
```

Q3. MySQL Student-Course Analytics Engine

CO-1, CO-4 | Bloom L6 Create | MySQL connection, create table, insert, select, where, order by, exception handling

Problem

Connect to MySQL, create Student and CourseRegistration tables if needed, load them from CSV files, and show students registered for a given course with SPI above a threshold, using parameterised queries and handling database errors.

Approach

- **Schema and indexes:** `Student(student_id PK, name, spi DECIMAL(4,2), INDEX(spi))` and `CourseRegistration(student_id FK, course_id, PRIMARY KEY(course_id, student_id))`. Because the primary key starts with course_id, `WHERE course_id = ?` is an index lookup, and each matching row reaches its student through the Student primary key. EXPLAIN in the tests confirms both index uses.
- **Streaming load:** each CSV is read row by row, validated in Python (id format, name length, SPI between 0 and 10, course id pattern) and sent in batches of 5,000 with `executemany()`. Python memory stays O(batch size) even for 5 million registrations.
- **Idempotent:** students are upserted (`ON DUPLICATE KEY UPDATE`) and registrations use `INSERT IGNORE`, so loading twice changes nothing. Duplicate registrations, and registrations for students that do not exist (foreign-key failures), are skipped and counted.
- Every value reaches MySQL as a `%s` parameter, never pasted into the SQL text. Results are fetched with `fetchmany()` in batches.
- **Errors:** wrong credentials, a missing database (created automatically), missing files and invalid input each give a clear one-line message instead of a traceback.

Complexity

Time O(r) to load r rows; the query is an index range scan on CourseRegistration plus one primary-key lookup per match, O(log n) each. **Space** O(batch size) in Python; MySQL stores the data.

Requires `pip install mysql-connector-python` and a MySQL server. Tested on MariaDB 10.11, which is MySQL-compatible; no MariaDB-only features are used.

Set-up used for testing: `CREATE DATABASE college; CREATE USER 'student'@'localhost' IDENTIFIED BY '...'; GRANT ALL ON college.* TO 'student'@'localhost';` The program creates the tables itself.

How to run

```
python ENROLLMENT_2_Q3.py --user student --database college student.csv registration.csv
PY101 8.0
```

Source code — ENROLLMENT_2_Q3.py

```python
"""
Q3. MySQL Student-Course Analytics Engine
Course : Programming with Python (202044504) - Assignment 2
CO     : CO-1, CO-4

1. Connects to MySQL (mysql-connector-python). If the database does not
   exist it is created.
2. Creates the tables if they do not exist (schema below).
3. Streams student.csv and registration.csv and inserts them in batches of
   5000 rows with executemany() and parameterised queries. Invalid rows are
   skipped and counted. Loading twice is safe (upsert / INSERT IGNORE).
4. Prints the students registered for COURSE_ID with SPI > THRESHOLD,
   sorted by SPI descending, then student id ascending.

Schema
    Student(student_id BIGINT PK, name VARCHAR(100), spi DECIMAL(4,2), INDEX(spi))
    CourseRegistration(student_id BIGINT FK -> Student, course_id VARCHAR(20),
                       PRIMARY KEY(course_id, student_id))
    The primary key starts with course_id, so "WHERE course_id = ?" is an
    index range scan (O(log n) to find the first row), and each matching
    registration finds its student through the Student primary key.

CSV files (header row required)
    student.csv       : student_id,name,spi
    registration.csv : student_id,course_id

Run
    pip install mysql-connector-python
    python ENROLLMENT_2_Q3.py --user student --database college \
           student.csv registration.csv PY101 8.0
    (password: --password, or env variable DB_PASSWORD, or typed at a prompt)
    --no-load skips step 3 and only runs the query.

Time  : load O(r) rows; query uses indexes as described above
Space : O(batch_size) in Python - neither file nor result is held in memory
"""

import argparse
import csv
import getpass
import os
import re
import sys
from decimal import Decimal, InvalidOperation

try:
    import mysql.connector
    from mysql.connector import errorcode
except ImportError:                                  # clear message instead of a traceback
    print("ERROR: mysql-connector-python is not installed. "
          "Run: pip install mysql-connector-python")
    sys.exit(1)

BATCH_SIZE = 5000
COURSE_RE = re.compile(r"^[A-Za-z0-9_-]{1,20}$")
ID_RE = re.compile(r"^\d{1,18}$")

SCHEMA = [
    """CREATE TABLE IF NOT EXISTS Student (
           student_id BIGINT UNSIGNED PRIMARY KEY,
           name       VARCHAR(100)   NOT NULL,
           spi        DECIMAL(4,2)   NOT NULL,
           CONSTRAINT chk_spi CHECK (spi BETWEEN 0 AND 10),
           INDEX idx_student_spi (spi)
       ) ENGINE=InnoDB""",
    """CREATE TABLE IF NOT EXISTS CourseRegistration (
           student_id BIGINT UNSIGNED NOT NULL,
           course_id  VARCHAR(20)     NOT NULL,
           PRIMARY KEY (course_id, student_id),
           INDEX idx_reg_student (student_id),
           CONSTRAINT fk_reg_student FOREIGN KEY (student_id)
               REFERENCES Student(student_id) ON DELETE CASCADE
```

```
        ) ENGINE=InnoDB""",
]

INSERT_STUDENT = ("INSERT INTO Student (student_id, name, spi) VALUES (%s, %s, %s) "
                  "ON DUPLICATE KEY UPDATE name = VALUES(name), spi = VALUES(spi)")
# INSERT IGNORE skips duplicate registrations and registrations of unknown
# students (foreign-key failures become warnings instead of aborting the batch)
INSERT_REG = "INSERT IGNORE INTO CourseRegistration (student_id, course_id) VALUES (%s, %s)"

QUERY = """SELECT s.student_id, s.name, s.spi, r.course_id
           FROM CourseRegistration AS r
           JOIN Student AS s ON s.student_id = r.student_id
           WHERE r.course_id = %s AND s.spi > %s
           ORDER BY s.spi DESC, s.student_id ASC"""


class DataError(Exception):
    """A CSV row is invalid; message says why."""


#---------------------------------------------------------------------------------------------validation
def parse_spi(text):
    try:
        spi = Decimal(text.strip())
    except InvalidOperation:
        raise DataError(f"SPI '{text}' is not a number") from None
    if not spi.is_finite() or not 0 <= spi <= 10:
        raise DataError(f"SPI {text} outside 0.0-10.0")
    return spi.quantize(Decimal("0.01"))


def student_row(row):
    if len(row) != 3:
        raise DataError(f"expected 3 columns, got {len(row)}")
    sid, name, spi = (x.strip() for x in row)
    if not ID_RE.match(sid):
        raise DataError(f"student_id '{sid}' must be 1-18 digits")
    if not 1 <= len(name) <= 100:
        raise DataError("name must be 1-100 characters")
    return int(sid), name, parse_spi(spi)


def registration_row(row):
    if len(row) != 2:
        raise DataError(f"expected 2 columns, got {len(row)}")
    sid, course = (x.strip() for x in row)
    if not ID_RE.match(sid):
        raise DataError(f"student_id '{sid}' must be 1-18 digits")
    if not COURSE_RE.match(course):
        raise DataError(f"course_id '{course}' is invalid")
    return int(sid), course.upper()


#---------------------------------------------------------------------------------------------database
def connect(args):
    password = args.password if args.password is not None else os.environ.get("DB_PASSWORD")
    if password is None:
        password = getpass.getpass("MySQL password: ")
    params = dict(host=args.host, port=args.port, user=args.user, password=password)
    try:
        return mysql.connector.connect(database=args.database, **params)
    except mysql.connector.Error as err:
        if err.errno != errorcode.ER_BAD_DB_ERROR:
            raise
        conn = mysql.connector.connect(**params)            # database missing -> create it
        cur = conn.cursor()
        cur.execute(f"CREATE DATABASE `{args.database}`")
        cur.close()
        conn.database = args.database
        return conn
```

```
145     def create_tables(conn):
146         cur = conn.cursor()
147         for statement in SCHEMA:
148             cur.execute(statement)
149         conn.commit()
150         cur.close()
151 
152 
153     def load_csv(conn, path, row_parser, sql, label, log=sys.stderr):
154         """Stream a CSV file into the database in batches. Returns (ok, rejected)."""
155         cur = conn.cursor()
156         batch, sent, inserted, rejected = [], 0, 0, 0
157 
158         def flush():
159             nonlocal inserted
160             cur.executemany(sql, batch)
161             inserted += max(cur.rowcount, 0)
162             conn.commit()
163             batch.clear()
164 
165         with open(path, newline="", encoding="utf-8-sig") as fh:
166             reader = csv.reader(fh)
167             if next(reader, None) is None:
168                 raise DataError(f"{path} is empty (header row missing)")
169             for line_no, row in enumerate(reader, start=2):
170                 if not any(cell.strip() for cell in row):
171                     continue
172                 try:
173                     batch.append(row_parser(row))
174                 except DataError as err:
175                     rejected += 1
176                     if rejected <= 5:
177                         print(f"  {label} line {line_no} rejected: {err}", file=log)
178                     continue
179                 sent += 1
180                 if len(batch) == BATCH_SIZE:
181                     flush()
182         if batch:
183             flush()
184         cur.close()
185         return sent, rejected, inserted
186 
187 
188     def students_above(conn, course_id, threshold):
189         """Generator over result rows, fetched in batches (O(batch) memory)."""
190         cur = conn.cursor()
191         cur.execute(QUERY, (course_id, threshold))
192         while rows := cur.fetchmany(BATCH_SIZE):
193             yield from rows
194         cur.close()
195 
196 
197     def fmt_spi(spi):
198         text = f"{spi:.2f}".rstrip("0")
199         return text + "0" if text.endswith(".") else text
200 
201 
202     #---------------------------------------------------------------------------------------------main
203     def build_args(argv=None):
204         ap = argparse.ArgumentParser(description="Student-course analytics on MySQL")
205         ap.add_argument("--host", default="localhost")
206         ap.add_argument("--port", type=int, default=3306)
207         ap.add_argument("--user", default="root")
208         ap.add_argument("--password")
209         ap.add_argument("--database", default="college")
210         ap.add_argument("--no-load", action="store_true", help="skip loading the CSV  files")
211         ap.add_argument("student_csv")
212         ap.add_argument("registration_csv")
213         ap.add_argument("course_id")
214         ap.add_argument("spi_threshold")
215         return ap.parse_args(argv)
216 
```

```
217
218 def run(args, out=print, log=sys.stderr):
219     course = args.course_id.strip().upper()
220     if not COURSE_RE.match(course):
221         raise DataError(f"course_id '{args.course_id}' is invalid")
222     threshold = parse_spi(args.spi_threshold)
223
224     conn = connect(args)
225     try:
226         create_tables(conn)
227         if not args.no_load:
228             s = load_csv(conn, args.student_csv, student_row, INSERT_STUDENT, "student.csv",   log)
229             r = load_csv(conn, args.registration_csv, registration_row, INSERT_REG,
230                          "registration.csv", log)
231             print(f"Loaded students: {s[0]} valid, {s[1]} rejected | "
232                   f"registrations: {r[0]} valid, {r[1]} rejected, "
233                   f"{r[0] - r[2]} skipped as duplicate/unknown student", file=log)
234         found = 0
235         for sid, name, spi, cid in students_above(conn, course, threshold):
236             out(f"{sid} {name} {fmt_spi(spi)} {cid}")
237             found += 1
238         if not found:
239             out("NO_MATCHING_STUDENTS")
240     finally:
241         conn.close()
242
243
244 def main():
245     args = build_args()
246     try:
247         run(args)
248     except DataError as err:
249         print(f"INPUT ERROR: {err}")
250         sys.exit(1)
251     except FileNotFoundError as err:
252         print(f"ERROR: file not found: {err.filename}")
253         sys.exit(1)
254     except mysql.connector.Error as err:
255         if err.errno == errorcode.ER_ACCESS_DENIED_ERROR:
256             print("DATABASE ERROR: wrong user name or password")
257         else:
258             print(f"DATABASE ERROR: {err.msg if hasattr(err, 'msg')  else err}")
259         sys.exit(1)
260
261
262 if __name__ == "__main__":
263     main()
```

Test cases — 9 of 9 passed

#	Test case	Input	Output (expected = actual)	Result
1	**Sample test: PY101, SPI > 8.0 (Chaitra has exactly 8.0)** sample	`student.csv, registration.csv (see Q3 folder)` `course_id = PY101, spi_threshold = 8.0`	`2201 Asha 9.1 PY101` `2207 Dev 8.4 PY101`	**PASS**

2	**Loading again is idempotent; threshold 0 lists every PY101 student**
	boundary
		`same files loaded a second time; PY101 0`	`2201 Asha 9.1 PY101`
			`2207 Dev 8.4 PY101`
			`2203 Chaitra 8.0 PY101`
			`2202 Bharat 7.9 PY101`
				PASS

3	**Boundary: threshold 10.0 -> nobody** boundary	`--no-load PY101 10`	`NO_MATCHING_STUDENTS`	**PASS**

#	Test case	Input	Output (expected = actual)	Result

4	**Invalid rows rejected, duplicate + unknown-student registrations skipped**
	custom
		`s . c s v :`
		`student_id,name,spi`
		`3001,Ravi,9.5`
		`3002,,8`
		`abc,Bad Id,7`
		`3003,Tina,11`
		`3004,Uma,8.75`
		`3005,Vik,9.50`
		`r . c s v :`
		`student_id,course_id`
		`3001,ML201`
		`3004,ml201`
		`3005,ML201`
		`3001,ML201`
		`9999,ML201`
		`3005,ML 201`
		`course ml201, spi 8`
			`3001 Ravi 9.5 ML201`
			`3005 Vik 9.5 ML201`
			`3004 Uma 8.75 ML201`
			`[stderr]`
			`student.csv line 3 rejected: name must be 1-100 characters`
			`student.csv line 4 rejected: student_id 'abc' must be 1-18 digits`
			`student.csv line 5 rejected: SPI 11 outside 0.0-10.0`
			`registration.csv line 7 rejected: course_id 'ML 201' is invalid`
			`Loaded students: 3 valid, 3 rejected | registrations: 5 valid, 1 rejected, 2 skipped as duplicate/unknown student`
				PASS

5	**Invalid: unsafe course id** invalid	`course_id = PY101;DROP`	`INPUT ERROR: course_id 'PY101;DROP' is invalid`	**PASS**
6	**Invalid: wrong password** invalid	`--password wrong`	`DATABASE ERROR: wrong user name or password`	**PASS**
7	**Large: load 1,000,000 students + 5,000,000 registrations, then query** performance	`random SPIs, 200 courses`	`finished in 992.11 s, 1210 rows returned`	**PASS**
8	**Query only on the large tables (indexed)** performance	`--no-load C007 9.5`	`finished in 0.40 s, 1210 rows`	**PASS**
9	**Query plan uses indexes (EXPLAIN)** custom	`EXPLAIN on the result query`	`r: type=ref key=PRIMARY; s: type=eq_ref key=PRIMARY`	**PASS**

Query plan uses indexes (EXPLAIN): r uses the PRIMARY KEY (course_id, ...) as a range/ref scan; s is looked up by its primary key (eq_ref).

Sample file — `student.csv`

```
student_id,name,spi
2201,Asha,9.1
2202,Bharat,7.9
2203,Chaitra,8.0
2205,Esha,6.5
2207,Dev,8.4
```

Sample file — `registration.csv`

```
student_id,course_id
2201,PY101
2201,DB102
2202,PY101
2203,PY101
2203,OS103
2205,DB102
2207,PY101
2207,OS103
```

Q4. SQL Join Query Builder and Validator

CO-1, CO-4 | Bloom L5 Evaluate | Database programming, joins, validation, SQL injection prevention, dictionaries

Problem

A command-line builder that turns a request (columns, filters, order, limit) into safe parameterised SQL joining Student, Course and CourseRegistration, rejects invalid columns and unsafe input before execution, and prints the SQL, its parameters and the result.

Approach

- **Whitelist dictionary:** public names such as `student.spi` map to (table alias, real column, type). Identifiers in the SQL come only from this dictionary; user text is never used as a table or column name, which is where parameters cannot help.
- **Values are always parameters** (`%s`). The test sends `PY101' OR '1'='1` : it is treated as a plain string and matches nothing.
- **Type-aware validation:** LIKE is allowed only on text columns and range operators only on numeric ones; numeric values must parse; LIMIT is an integer from 1 to 1000; direction is ASC or DESC; values are at most 100 characters with no control characters; at most 10 filters. Any violation prints `SQL_REJECTED: reason` and nothing runs.
- **Minimal joins:** only the tables actually used are included. Student-only requests query Student alone; mixing student and course columns joins through CourseRegistration.
- **Three input forms** build the same dictionary: JSON, the one-line form used in the paper's sample, and an interactive menu (`--menu`). `--init-demo` creates the tables with sample data, and `--dry-run` validates without connecting.

Complexity

Time O(number of requested fields and filters) for validation and SQL generation; execution depends on the query plan (indexes from Q3). **Space** O(limit) result rows.

> The sample output in the paper shows only SQL_OK and the rows. The format line also asks for the SQL template and parameter list, so both are printed between them.

How to run

```
python ENROLLMENT_2_Q4.py --user student --database college --init-demo
python ENROLLMENT_2_Q4.py --user student --database college request.json
```

Source code — ENROLLMENT_2_Q4.py

```python
"""
Q4. SQL Join Query Builder and Validator
Course : Programming with Python (202044504) - Assignment 2
CO     : CO-1, CO-4

Turns a query request (JSON, a one-line text form, or an interactive menu)
into a SAFE parameterised SELECT over Student, Course and CourseRegistration,
prints the SQL template and the parameter list, then runs it.

Safety rules (checked before anything reaches the database)
  * Columns, tables and the ORDER BY column come ONLY from a whitelist
    dictionary. User text is never pasted into SQL as an identifier.
  * Every value is sent as a %s parameter; the driver escapes it, so input
    such as  PY101' OR '1'='1  is just a string that matches nothing.
  * Operators: = != < <= > >= LIKE BETWEEN. LIKE only on text columns,
    range operators only on numeric columns; values must have the right type.
  * Direction must be ASC or DESC; LIMIT must be an integer 1..1000;
    values: max 100 characters, no control characters; max 10 filters.
  * Only the tables that are needed are joined.
Any violation prints  SQL_REJECTED: <reason>  and nothing is executed.

Request formats
  JSON : {"columns": ["student.name", "course.name"],
          "where": [{"column": "course.id", "op": "=", "value": "PY101"}],
          "order": {"column": "student.spi", "direction": "DESC"},
          "limit": 2}
  Text : columns: student.name, course.name; where: course.id=PY101;
         order: student.spi DESC; limit: 2
         (several conditions: "a=1 AND b>2"; range: "student.spi BETWEEN 8 TO 9")
  Menu : --menu  asks for each part step by step

Run
  python ENROLLMENT_2_Q4.py --user student --database college --init-demo     (once)
  python ENROLLMENT_2_Q4.py --user student --database college request.json
  echo "columns: ...; limit: 2" | python ENROLLMENT_2_Q4.py --user student -
  --dry-run validates and prints the SQL without connecting.

Time  : validation O(number of requested fields and filters); query: index dependent
Space : O(limit) result rows
"""

import argparse
import json
import os
import re
import sys
from decimal import Decimal, InvalidOperation

MAX_FILTERS = 10
MAX_VALUE_LEN = 100

# public name -> (table alias, SQL column, type)
COLUMNS = {
    "student.id":       ("s", "student_id", "int"),
    "student.name":     ("s", "name",       "text"),
    "student.spi":      ("s", "spi",        "num"),
    "course.id":        ("c", "course_id",  "text"),
    "course.name":      ("c", "name",       "text"),
    "course.credits":   ("c", "credits",    "int"),
}
TABLES = {"s": "Student AS s", "c": "Course AS c"}
OPERATORS = {"=", "!=", "<", "<=", ">", ">=", "LIKE", "BETWEEN"}
RANGE_OPS = {"<", "<=", ">", ">=", "BETWEEN"}
CONTROL_RE = re.compile(r"[\x00-\x1f\x7f]")
COND_RE = re.compile(r"^\s*([\w.]+)\s*(>=|<=|!=|=|<|>|\bLIKE\b|\bBETWEEN\b)\s*(.+?)\s*$",
                     re.IGNORECASE)


class QueryRejected(Exception):
    """The request is invalid or unsafe; nothing will be executed."""
```

```
# ============================================================ validation
def check_column(name, purpose):
    if not isinstance(name, str) or name.strip().lower() not in COLUMNS:
        raise QueryRejected(f"unknown {purpose} column {name!r}; allowed: {', '.join(COLUMNS)}")
    return name.strip().lower()


def check_value(value, col_type, column):
    if isinstance(value, bool) or not isinstance(value, (str, int, float)):
        raise QueryRejected(f"value for {column} must be a string or number")
    if isinstance(value, str):
        if len(value) > MAX_VALUE_LEN:
            raise QueryRejected(f"value for {column} is longer than {MAX_VALUE_LEN} characters")
        if CONTROL_RE.search(value):
            raise QueryRejected(f"value for {column} contains control characters")
    if col_type == "text":
        return str(value)
    try:
        number = Decimal(str(value).strip())
    except InvalidOperation:
        raise QueryRejected(f"value {value!r} for {column} must be numeric") from None
    if not number.is_finite():
        raise QueryRejected(f"value for {column} must be a finite number")
    if col_type == "int":
        if number != number.to_integral_value():
            raise QueryRejected(f"value for {column} must be an integer")
        return int(number)
    return number


def build_condition(cond, params):
    """Return SQL text for one filter and append its parameters."""
    if not isinstance(cond, dict):
        raise QueryRejected("each filter must be an object with column, op, value")
    column = check_column(cond.get("column"), "filter")
    alias, sql_col, col_type = COLUMNS[column]
    op = str(cond.get("op", "")).strip().upper()
    if op not in OPERATORS:
        raise QueryRejected(f"operator {cond.get('op')!r} is not allowed")
    if op == "LIKE" and col_type != "text":
        raise QueryRejected(f"LIKE is only allowed on text columns, not {column}")
    if op in RANGE_OPS and col_type == "text":
        raise QueryRejected(f"range operator {op} is only allowed on numeric columns")
    target = f"{alias}.{sql_col}"
    if op == "BETWEEN":
        value = cond.get("value")
        if not isinstance(value, (list, tuple)) or len(value) != 2:
            raise QueryRejected("BETWEEN needs a [low, high] pair")
        low, high = (check_value(v, col_type, column) for v in value)
        if low > high:
            raise QueryRejected("BETWEEN low value is greater than high value")
        params += [low, high]
        return f"{target} BETWEEN %s AND %s"
    params.append(check_value(cond.get("value"), col_type, column))
    sql_op = "<>" if op == "!=" else op
    return f"{target} {sql_op} %s"


def build_query(request):
    """Validate a request dict and return (sql_template, params, headers)."""
    if not isinstance(request, dict):
        raise QueryRejected("request must be a JSON object")
    unknown = set(request) - {"columns", "where", "order", "limit"}
    if unknown:
        raise QueryRejected(f"unknown request keys: {', '.join(sorted(unknown))}")

    columns = request.get("columns")
    if not isinstance(columns, list) or not columns:
        raise QueryRejected("columns must be a non-empty list")
    columns = [check_column(c, "select") for c in columns]
    if len(set(columns)) != len(columns):
        raise QueryRejected("a column is selected twice")
```

```
145
146         where = request.get("where", [])
147         if isinstance(where, dict):
148             where = [where]
149         if not isinstance(where, list) or len(where) > MAX_FILTERS:
150             raise QueryRejected(f"where must be a list of at most {MAX_FILTERS} filters")
151
152         order = request.get("order")
153         order_sql = ""
154         order_col = None
155         if order is not None:
156             if not isinstance(order, dict):
157                 raise QueryRejected("order must be an object with column and direction")
158             order_col = check_column(order.get("column"), "order")
159             direction = str(order.get("direction", "ASC")).strip().upper()
160             if direction not in ("ASC", "DESC"):
161                 raise QueryRejected("direction must be ASC or DESC")
162             alias, sql_col, _ = COLUMNS[order_col]
163             order_sql = f" ORDER BY {alias}.{sql_col} {direction}"
164
165         limit = request.get("limit", 100)
166         if isinstance(limit, bool) or not isinstance(limit, int) or not 1 <= limit <= 1000:
167             raise QueryRejected("limit must be an integer between 1 and 1000")
168
169         params = []
170         conditions = [build_condition(c, params) for c in where]
171
172         # join only what is needed
173         used = {COLUMNS[c][0] for c in columns}
174         used |= {COLUMNS[check_column(c.get("column"), "filter")][0] for c in where}
175         if order_col:
176             used.add(COLUMNS[order_col][0])
177         if used == {"s", "c"}:
178             from_sql = ("Student AS s JOIN CourseRegistration AS r ON r.student_id = s.student_id "
179                         "JOIN Course AS c ON c.course_id = r.course_id")
180         else:
181             from_sql = TABLES[used.pop()]
182
183         select = ", ".join(f"{COLUMNS[c][0]}.{COLUMNS[c][1]}" for c in columns)
184         sql = f"SELECT {select} FROM {from_sql}"
185         if conditions:
186             sql += " WHERE " + " AND ".join(conditions)
187         sql += order_sql + " LIMIT %s"
188         params.append(limit)
189         return sql, params, columns
190
191
192     # ============================================================ input forms
193     def parse_text_request(text):
194         """Parse 'columns: a, b; where: x=1 AND y>2; order: c DESC; limit: 5'."""
195         request = {}
196         for part in filter(None, (p.strip() for p in text.split(";"))):
197             if ":" not in part:
198                 raise QueryRejected(f"cannot understand '{part}'")
199             key, value = (x.strip() for x in part.split(":", 1))
200             key = key.lower()
201             if key == "columns":
202                 request["columns"] = [c.strip() for c in value.split(",") if c.strip()]
203             elif key == "where":
204                 request["where"] = [parse_text_condition(c)
205                                     for c in re.split(r"\s+AND\s+", value, flags=re.I) if c.strip()]
206             elif key == "order":
207                 bits = value.split()
208                 if len(bits) not in (1, 2):
209                     raise QueryRejected("order must be: column [ASC|DESC]")
210                 request["order"] = {"column": bits[0], "direction": bits[1] if len(bits) == 2 else "ASC"}
211             elif key == "limit":
212                 try:
213                     request["limit"] = int(value)
214                 except ValueError:
215                     raise QueryRejected("limit must be an integer between 1 and 1000") from None
216             else:
```

```
                raise QueryRejected(f"unknown section '{key}'")
        return request


def parse_text_condition(text):
    m = COND_RE.match(text)
    if not m:
        raise QueryRejected(f"cannot understand condition '{text}'")
    column, op, value = m.group(1), m.group(2).upper(), m.group(3).strip()
    if len(value) >= 2 and value[0] == value[-1] and value[0] in "'\"":
        value = value[1:-1]
    if op == "BETWEEN":
        bits = re.split(r"\s+TO\s+", value, flags=re.I)
        if len(bits) != 2:
            raise QueryRejected("BETWEEN must be written as: column BETWEEN low TO high")
        value = bits
    return {"column": column, "op": op, "value": value}


def parse_request(text):
    text = text.strip()
    if not text:
        raise QueryRejected("empty request")
    if text.startswith("{"):
        try:
            return json.loads(text)
        except json.JSONDecodeError as err:
            raise QueryRejected(f"invalid JSON: {err.msg}") from None
    return parse_text_request(text)


def menu_request(ask=input, out=print):
    """Interactive menu: builds the same dict as the JSON form."""
    out("Available columns: " + ", ".join(COLUMNS))
    cols = ask("Columns to show (comma separated): ")
    request = {"columns": [c.strip() for c in cols.split(",") if c.strip()], "where": []}
    while True:
        cond = ask("Filter (e.g. course.id=PY101), blank to finish: ").strip()
        if not cond:
            break
        request["where"].append(parse_text_condition(cond))
    order = ask("Order by column [ASC|DESC] (blank for none): ").strip()
    if order:
        bits = order.split()
        request["order"] = {"column": bits[0], "direction": bits[1] if len(bits) > 1 else "ASC"}
    limit = ask("Limit (1-1000) [100]: ").strip()
    try:
        request["limit"] = int(limit) if limit else 100
    except ValueError:
        raise QueryRejected("limit must be an integer between 1 and 1000") from None
    return request


# ============================================================ database
DEMO_SCHEMA = [
    """CREATE TABLE IF NOT EXISTS Student (
            student_id BIGINT UNSIGNED PRIMARY KEY, name VARCHAR(100) NOT NULL,
            spi DECIMAL(4,2) NOT NULL, INDEX idx_student_spi (spi)) ENGINE=InnoDB""",
    """CREATE TABLE IF NOT EXISTS Course (
            course_id VARCHAR(20) PRIMARY KEY, name VARCHAR(100) NOT NULL,
            credits TINYINT UNSIGNED NOT NULL) ENGINE=InnoDB""",
    """CREATE TABLE IF NOT EXISTS CourseRegistration (
            student_id BIGINT UNSIGNED NOT NULL, course_id VARCHAR(20) NOT NULL,
            PRIMARY KEY (course_id, student_id), INDEX idx_reg_student (student_id),
            FOREIGN KEY (student_id) REFERENCES Student(student_id) ON DELETE CASCADE
        ) ENGINE=InnoDB""",
]
DEMO_STUDENTS = [(2201, "Asha", "9.10"), (2202, "Bharat", "7.90"), (2203, "Chaitra", "8.00"),
                 (2205, "Esha", "6.50"), (2207, "Dev", "8.40")]
DEMO_COURSES = [("PY101", "Programming with Python", 4), ("DB102", "Database Management Systems", 4),
                ("OS103", "Operating Systems", 3)]
DEMO_REGS = [(2201, "PY101"), (2201, "DB102"), (2202, "PY101"), (2203, "PY101"),
```

```
                (2203, "OS103"), (2205, "DB102"), (2207, "PY101"), (2207, "OS103")]


def connect(args):
    import mysql.connector
    password = args.password if args.password is not None else os.environ.get("DB_PASSWORD")
    if password is None:
        import getpass
        password = getpass.getpass("MySQL password: ")
    return mysql.connector.connect(host=args.host, port=args.port, user=args.user,
                                   password=password, database=args.database)


def init_demo(conn):
    cur = conn.cursor()
    for stmt in DEMO_SCHEMA:
        cur.execute(stmt)
    cur.executemany("INSERT INTO Student VALUES (%s,%s,%s) ON DUPLICATE KEY UPDATE "
                    "name=VALUES(name), spi=VALUES(spi)", DEMO_STUDENTS)
    cur.executemany("INSERT INTO Course VALUES (%s,%s,%s) ON DUPLICATE KEY UPDATE "
                    "name=VALUES(name), credits=VALUES(credits)", DEMO_COURSES)
    cur.executemany("INSERT IGNORE INTO CourseRegistration VALUES (%s,%s)", DEMO_REGS)
    conn.commit()
    cur.close()


def execute(conn, sql, params):
    cur = conn.cursor()
    cur.execute(sql, params)
    rows = cur.fetchall()              # at most LIMIT (<= 1000) rows
    cur.close()
    return rows


def fmt(value):
    if isinstance(value, Decimal):
        text = f"{value:.2f}".rstrip("0")
        return text + "0" if text.endswith(".") else text
    return str(value)


def print_table(headers, rows, out=print):
    if not rows:
        out("(no rows)")
        return
    for row in rows:
        out(" ".join(fmt(v) for v in row))
    out(f"({len(rows)} row{'s' if len(rows) != 1 else ''}; columns: {', '.join(headers)})")


# ============================================================ main
def build_args(argv=None):
    ap = argparse.ArgumentParser(description="Safe SQL join query builder")
    ap.add_argument("request", nargs="?", default="-",
                    help="JSON/text request file, or - for standard input")
    ap.add_argument("--menu", action="store_true", help="build the request interactively")
    ap.add_argument("--dry-run", action="store_true", help="only validate and print the SQL")
    ap.add_argument("--init-demo", action="store_true", help="create tables + sample data")
    ap.add_argument("--host", default="localhost")
    ap.add_argument("--port", type=int, default=3306)
    ap.add_argument("--user", default="root")
    ap.add_argument("--password")
    ap.add_argument("--database", default="college")
    return ap.parse_args(argv)


def main(argv=None):
    args = build_args(argv)
    if args.init_demo:
        conn = connect(args)
        init_demo(conn)
        conn.close()
```

```
            print("Demo tables and data are ready.")
            return
        try:
            if args.menu:
                request = menu_request()
            else:
                if args.request == "-":
                    text = sys.stdin.read()
                else:
                    with open(args.request, encoding="utf-8") as fh:
                        text = fh.read()
                request = parse_request(text)
            sql, params, headers = build_query(request)
        except QueryRejected as err:
            print(f"SQL_REJECTED: {err}")
            sys.exit(2)
        except OSError as err:
            print(f"ERROR: cannot read request: {err}")
            sys.exit(1)

        print("SQL_OK")
        print(f"SQL: {sql}")
        print("PARAMS: [" + ", ".join(str(p) if isinstance(p, (int, Decimal)) else repr(p)
                                       for p in params) + "]")
        if args.dry_run:
            return
        try:
            import mysql.connector
        except ImportError:
            print("ERROR: mysql-connector-python is not installed (pip install mysql-connector-python)")
            sys.exit(1)
        try:
            conn = connect(args)
            try:
                print_table(headers, execute(conn, sql, params))
            finally:
                conn.close()
        except mysql.connector.Error as err:
            print(f"DATABASE ERROR: {getattr(err, 'msg', err)}")
            sys.exit(1)


if __name__ == "__main__":
    main()
```

Test cases — 14 of 14 passed

#	Test case	Input	Output (expected = actual)	Result

1	**Sample test (one-line text request)**
	sample
		`columns: student.name, course.name; where: course.id=PY101; order: student.spi DESC; limit: 2`	`SQL_OK`
			`SQL: SELECT s.name, c.name FROM Student AS s JOIN CourseRegistration AS r ON r.student_id = s.student_id JOIN Course AS c ON c.course_id = r.course_id WHERE c.course_id = %s ORDER BY s.spi DESC LIMIT %s`
			`PARAMS: ['PY101', 2]`
			`Asha Programming with Python`
			`Dev Programming with Python`
			`(2 rows; columns: student.name, course.name)`
				PASS

2	**Same request as JSON**
	custom
		`{"columns": ["student.name", "course.name"],`
		`"where": [{"column": "course.id", "op": "=", "value": "PY101"}],`
		`"order": {"column": "student.spi", "direction": "DESC"},`
		`"limit": 2}`
			`SQL_OK`
			`SQL: SELECT s.name, c.name FROM Student AS s JOIN CourseRegistration AS r ON r.student_id = s.student_id JOIN Course AS c ON c.course_id = r.course_id WHERE c.course_id = %s ORDER BY s.spi DESC LIMIT %s`
			`PARAMS: ['PY101', 2]`
			`Asha Programming with Python`
			`Dev Programming with Python`
			`(2 rows; columns: student.name, course.name)`
				PASS

#	Test case	Input	Output (expected = actual)	Result

3	**SQL injection attempt in a value is only data**
	custom
		`columns:student.name,course.name; where: course.id="PY101' OR '1'='1"; limit: 5`	`SQL_OK`
			`SQL: SELECT s.name, c.name FROM Student AS s JOIN CourseRegistration AS r ON r.student_id = s.student_id JOIN Course AS c ON c.course_id = r.course_id WHERE c.course_id = %s LIMIT %s`
			`PARAMS: ["PY101' OR '1'='1", 5]`
			`(no rows)`
				PASS

4	**BETWEEN on one table only (no join needed)**
	custom
		`columns: student.id, student.name, student.spi; where: student.spi BETWEEN 8 TO 9.5; order: student.spi DESC; limit: 10`	`SQL_OK`
			`SQL: SELECT s.student_id, s.name, s.spi FROM Student AS s WHERE s.spi BETWEEN %s AND %s ORDER BY s.spi DESC LIMIT %s`
			`PARAMS: [8, 9.5, 10]`
			`2201 Asha 9.1`
			`2207 Dev 8.4`
			`2203 Chaitra 8.0`
			`(3 rows; columns: student.id, student.name, student.spi)`
				PASS

5	**LIKE filter + ASC order**
	custom
		`{"columns":["course.id","course.name"],"where":[{"column":"course.name","op":"LIKE","value":"%Systems"}],"order":{"column":"course.id","direction":"asc"},"limit":5}`	`SQL_OK`
			`SQL: SELECT c.course_id, c.name FROM Course AS c WHERE c.name LIKE %s ORDER BY c.course_id ASC LIMIT %s`
			`PARAMS: ['%Systems', 5]`
			`DB102 Database Management Systems`
			`OS103 Operating Systems`
			`(2 rows; columns: course.id, course.name)`
				PASS

6	**Rejected: unknown select column 'student.password'** invalid	`columns: student.password; limit: 2`	`SQL_REJECTED:unknownselectcolumn 'student.password'; allowed: student.id, student.name, student.spi, course.id, course.name, course.credits`	**PASS**
7	**Rejected: cannot understand 'DROP TABLE Student'** invalid	`columns: student.name; DROP TABLE Student; limit: 2`	`SQL_REJECTED: cannot understand 'DROP TABLE Student'`	**PASS**
8	**Rejected: direction must be ASC or DESC** invalid	`columns: student.name; order: student.spi DESC--; limit: 2`	`SQL_REJECTED: direction must be ASC or DESC`	**PASS**
9	**Rejected: limit must be an integer between 1 and 1000** invalid	`columns: student.name; limit: 1001`	`SQL_REJECTED: limit must be an integer between 1 and 1000`	**PASS**
10	**Rejected: limit must be an integer between 1 and 1000** invalid	`columns: student.name; limit: 0`	`SQL_REJECTED: limit must be an integer between 1 and 1000`	**PASS**
11	**Rejected: value 'abc' for student.spi must be numeric** invalid	`columns: student.name; where: student.spi=abc`	`SQL_REJECTED: value 'abc' for student.spi must be numeric`	**PASS**
12	**Rejected: LIKE is only allowed on text columns, not studen** invalid	`columns: student.name; where: student.spi LIKE 9%`	`SQL_REJECTED: LIKE is only allowed on text columns, not student.spi`	**PASS**
13	**Rejected: value for student.id must be a string or number** invalid	`{"columns":["student.name"],"where":[{"column":"student.id","op":"=","value":[1]}]}`	`SQL_REJECTED: value for student.id must be a string or number`	**PASS**

#	Test case	Input	Output (expected = actual)	Result
14	**Menu-driven input** boundary	--menu answers: student.id, student.name course.id=OS103 student.id ASC 5	SQL_OK SQL: SELECT s.student_id, s.name FROM Student AS s JOIN CourseRegistration AS r ON r.student_id = s.student_id JOIN Course AS c ON c.course_id = r.course_id WHERE c.course_id = %s ORDER BY s.student_id ASC LIMIT %s PARAMS: ['OS103', 5] 2203 Chaitra 2207 Dev (2 rows; columns: student.id, student.name)	**PASS**

Q5. Tkinter Contact Manager with MySQL Backend

CO-1, CO-3, CO-4, CO-5 | Bloom L6 Create | Tkinter, MySQL, regex validation, CRUD, event-driven programming

Problem

A Tkinter contact manager storing Contact(id, name, email, phone, category, notes) in MySQL, with create, update, delete, search by name/email/phone, sorted display, regex email validation and duplicate-email rejection, searchable at 10,000+ contacts.

Approach

- **Layers:** `ContactRepository` holds all SQL, validation and custom exceptions (ValidationError, DuplicateEmailError, NotFoundError), and is tested without a GUI. `ContactApp` only handles widgets and events.
- **Widgets:** Entry (name, email, phone, search), Text (notes), Combobox (category), Radiobutton (search field), Checkbutton (contains mode), Listbox with Scrollbar (contacts), Menu (File and Sort), Buttons, plus message boxes for errors.
- **Validation:** emails must match `^[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+(\.[A-Za-z0-9-]+)*\.[A-Za-z]{2,}$` with no '..'. Names and 10 to 15 digit phone numbers are also checked with regex. Emails are stored in lower case, and a UNIQUE index rejects duplicates even if two users save at the same moment.
- **Search:** 'starts with' uses `LIKE 'abc%'` on indexed columns, so EXPLAIN shows a range scan, O(log n + k). 'Contains' uses `LIKE '%abc%'`, which is a documented O(n) full scan. User text is escaped, so % and _ are matched literally. The list shows at most 500 rows with the total count, so it never freezes.
- **Sorting:** the Sort menu chooses name A to Z, name Z to A, newest first, or category. The ORDER BY text comes from a fixed dictionary, never from user input.

Complexity

Time O(log n + k) for indexed prefix search, O(n) for 'contains' (documented); insert, update and delete are O(log n) through the indexes. **Space** O(result size) in Python, at most 500 rows displayed.

Requires Tkinter and `mysql-connector-python`. The password can be given with --password, the DB_PASSWORD environment variable, or a password dialog that appears at start-up.

How to run

```
python ENROLLMENT_2_Q5.py --user student --database college
```

Source code — ENROLLMENT_2_Q5.py

```python
"""
Q5. Tkinter Contact Manager with MySQL Backend
Course : Programming with Python (202044504) - Assignment 2
CO     : CO-1, CO-3, CO-4, CO-5

Features: create, update, delete and search contacts (by name, email or
phone), sorted display, regex validation, duplicate-email rejection.

Database table
    Contact(id INT AUTO_INCREMENT PK, name VARCHAR(100), email VARCHAR(254) UNIQUE,
            phone VARCHAR(16), category VARCHAR(20), notes TEXT,
            INDEX(name), INDEX(phone))
    The UNIQUE index on email also makes email search fast.

Search complexity
    "Starts with" search uses  LIKE 'abc%'  on an indexed column, so MySQL does
    a B-tree range scan: O(log n + k) for k results.
    "Contains" search (checkbox) uses  LIKE '%abc%', which cannot use the index:
    O(n) full scan. Documented trade-off; fine for 10,000+ contacts.
    The list shows at most 500 rows, so the GUI never freezes on huge results.

Structure
    ContactRepository - all SQL (parameterised), validation, custom exceptions
    ContactApp        - Tkinter GUI; only calls ContactRepository

Widgets: Entry, Text, Combobox, Radiobutton, Checkbutton, Listbox (+Scrollbar),
         Menu, Button, LabelFrame, Label, messagebox, simpledialog

Run
    pip install mysql-connector-python
    python ENROLLMENT_2_Q5.py --user student --database college
    (password: --password, env DB_PASSWORD, or asked in a dialog)
"""

import argparse
import os
import re
import sys

EMAIL_RE = re.compile(r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+(\.[A-Za-z0-9-]+)*\.[A-Za-z]{2,}$")
NAME_RE = re.compile(r"^[A-Za-z][A-Za-z .'-]{0,99}$")
PHONE_RE = re.compile(r"^\+?\d{10,15}$")
CATEGORIES = ("Faculty", "Student", "Staff", "Family", "Friend", "Other")
SEARCH_FIELDS = {"name": "name", "email": "email", "phone": "phone"}
SORTS = {"Name A-Z": "name ASC, id ASC", "Name Z-A": "name DESC, id DESC",
         "Newest first": "id DESC", "Category": "category ASC, name ASC"}
MAX_ROWS = 500

SCHEMA = """CREATE TABLE IF NOT EXISTS Contact (
    id       INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    name     VARCHAR(100) NOT NULL,
    email    VARCHAR(254) NOT NULL,
    phone    VARCHAR(16),
    category VARCHAR(20)  NOT NULL DEFAULT 'Other',
    notes    TEXT,
    UNIQUE KEY uq_contact_email (email),
    INDEX idx_contact_name (name),
    INDEX idx_contact_phone (phone)
) ENGINE=InnoDB"""


class ContactError(Exception):
    """Base class: message is shown to the user."""


class ValidationError(ContactError):
    pass


class DuplicateEmailError(ContactError):
    pass

```

```
73
74  class NotFoundError(ContactError):
75      pass
76
77
78  def validate(name, email, phone, category, notes):
79      """Clean and check one contact; returns the cleaned tuple."""
80      name = " ".join(name.split())
81      email = email.strip().lower()
82      phone = re.sub(r"[\s-]", "", phone)
83      notes = notes.strip()
84      if not NAME_RE.match(name):
85          raise ValidationError("Name must start with a letter (letters, spaces, . ' - ; max 100).")
86      if len(email) > 254 or ".." in email or not EMAIL_RE.match(email):
87          raise ValidationError(f"'{email}' is not a valid email address.")
88      if phone and not PHONE_RE.match(phone):
89          raise ValidationError("Phone must have 10-15 digits (optional leading +).")
90      if category not in CATEGORIES:
91          raise ValidationError(f"Category must be one of: {', '.join(CATEGORIES)}.")
92      if len(notes) > 2000:
93          raise ValidationError("Notes can be at most 2000 characters.")
94      return name, email, phone or None, category, notes
95
96
97  def like_prefix(text):
98      """Escape LIKE wildcards so user text is matched literally."""
99      return text.replace("\\", "\\\\").replace("%", "\\%").replace("_", "\\_")
100
101
102 class ContactRepository:
103     def __init__(self, conn):
104         self.conn = conn
105         cur = conn.cursor()
106         cur.execute(SCHEMA)
107         conn.commit()
108         cur.close()
109
110     def _run(self, sql, params=(), fetch=False):
111         import mysql.connector
112         cur = self.conn.cursor()
113         try:
114             cur.execute(sql, params)
115             rows = cur.fetchall() if fetch else None
116             self.conn.commit()
117             return rows if fetch else cur
118         except mysql.connector.IntegrityError as err:
119             self.conn.rollback()
120             if err.errno == 1062:                          # ER_DUP_ENTRY
121                 raise DuplicateEmailError("A contact with this email already exists.") from None
122             raise
123         finally:
124             if fetch:
125                 cur.close()
126
127     def add(self, name, email, phone="", category="Other", notes=""):
128         data = validate(name, email, phone, category, notes)
129         cur = self._run("INSERT INTO Contact (name, email, phone, category, notes) "
130                         "VALUES (%s, %s, %s, %s, %s)", data)
131         new_id = cur.lastrowid
132         cur.close()
133         return new_id
134
135     def update(self, contact_id, name, email, phone="", category="Other", notes=""):
136         data = validate(name, email, phone, category, notes)
137         if not self.get(contact_id):
138             raise NotFoundError(f"Contact {contact_id} no longer exists.")
139         self._run("UPDATE Contact SET name=%s, email=%s, phone=%s, category=%s, notes=%s "
140                   "WHERE id=%s", (*data, contact_id)).close()
141
142     def delete(self, contact_id):
143         cur = self._run("DELETE FROM Contact WHERE id=%s", (contact_id,))
144         deleted = cur.rowcount
```

```
        cur.close()
        if not deleted:
            raise NotFoundError(f"Contact {contact_id} does not exist.")

    def get(self, contact_id):
        rows = self._run("SELECT id, name, email, phone, category, notes FROM Contact "
                         "WHERE id=%s", (contact_id,), fetch=True)
        return rows[0] if rows else None

    def search(self, text="", field="name", contains=False, sort="Name A-Z", limit=MAX_ROWS):
        """Return (rows, total_matches). Column and sort come from whitelists only."""
        column = SEARCH_FIELDS[field]
        order = SORTS[sort]
        text = text.strip()
        where, params = "", []
        if text:
            if field == "email":
                text = text.lower()
            if field == "phone":
                text = re.sub(r"[\s-]", "", text)
            pattern = like_prefix(text) + "%"
            if contains:
                pattern = "%" + pattern
            where, params = f" WHERE {column} LIKE %s", [pattern]
        total = self._run(f"SELECT COUNT(*) FROM Contact{where}", params, fetch=True)[0][0]
        rows = self._run(f"SELECT id, name, email, phone, category FROM Contact{where} "
                         f"ORDER BY {order} LIMIT %s", params + [limit], fetch=True)
        return rows, total


# ================================================================== GUI
def run_gui(repo):
    import tkinter as tk
    from tkinter import messagebox, ttk

    class ContactApp:
        def __init__(self, root):
            self.root = root
            self.repo = repo
            self.selected_id = None
            self.row_ids = []
            root.title("Contact Manager")
            root.geometry("1100x560")
            root.minsize(880, 500)
            self.sort_var = tk.StringVar(value="Name A-Z")
            self._menu()
            self._form()
            self._list()
            self.refresh()

        def _menu(self):
            bar = tk.Menu(self.root)
            sort_menu = tk.Menu(bar, tearoff=False)
            for label in SORTS:
                sort_menu.add_radiobutton(label=label, value=label, variable=self.sort_var,
                                          command=self.refresh)
            file_menu = tk.Menu(bar, tearoff=False)
            file_menu.add_command(label="New contact", command=self.clear)
            file_menu.add_separator()
            file_menu.add_command(label="Exit", command=self.root.destroy)
            bar.add_cascade(label="File", menu=file_menu)
            bar.add_cascade(label="Sort", menu=sort_menu)
            self.root.config(menu=bar)

        def _form(self):
            box = ttk.LabelFrame(self.root, text="Contact details", padding=10)
            box.pack(side="left", fill="y", padx=(10, 5), pady=10)
            self.vars = {k: tk.StringVar() for k in ("name", "email", "phone")}
            for r, (key, label) in enumerate((("name", "Name"), ("email", "Email"), ("phone",   "Phone"))):
                ttk.Label(box, text=label).grid(row=r, column=0, sticky="w", pady=3)
                ttk.Entry(box, textvariable=self.vars[key], width=32).grid(row=r, column=1, pady=3)
            ttk.Label(box, text="Category").grid(row=3, column=0, sticky="w", pady=3)
```

```
217                 self.category = ttk.Combobox(box, values=CATEGORIES, state="readonly", width=29)
218                 self.category.set("Other")
219                 self.category.grid(row=3, column=1, pady=3, sticky="w")
220                 ttk.Label(box, text="Notes").grid(row=4, column=0, sticky="nw", pady=3)
221                 self.notes = tk.Text(box, width=32, height=7, wrap="word")
222                 self.notes.grid(row=4, column=1, pady=3)
223                 btns = ttk.Frame(box)
224                 btns.grid(row=5, column=0, columnspan=2, pady=(10, 0), sticky="ew")
225                 for text, cmd in (("Add", self.add), ("Update", self.update),
226                                   ("Delete", self.delete), ("Clear", self.clear)):
227                     ttk.Button(btns, text=text, command=cmd, width=8).pack(side="left", padx=2)
228
229             def _list(self):
230                 right = ttk.Frame(self.root, padding=(5, 10, 10, 10))
231                 right.pack(side="left", fill="both", expand=True)
232                 top = ttk.Frame(right)
233                 top.pack(fill="x")
234                 ttk.Label(top, text="Search").pack(side="left")
235                 self.search_var = tk.StringVar()
236                 self.search_var.trace_add("write", lambda *_: self.refresh())
237                 ttk.Entry(top, textvariable=self.search_var, width=24).pack(side="left", padx=5)
238                 self.field_var = tk.StringVar(value="name")
239                 for value in ("name", "email", "phone"):
240                     ttk.Radiobutton(top, text=value.title(), value=value, variable=self.field_var,
241                                     command=self.refresh).pack(side="left")
242                 self.contains_var = tk.BooleanVar(value=False)
243                 ttk.Checkbutton(top, text="Contains", variable=self.contains_var,
244                                 command=self.refresh).pack(side="left", padx=6)
245
246                 frame = ttk.Frame(right)
247                 frame.pack(fill="both", expand=True, pady=(8, 0))
248                 self.listbox = tk.Listbox(frame, font=("DejaVu Sans Mono", 9), activestyle="none")
249                 scroll = ttk.Scrollbar(frame, orient="vertical", command=self.listbox.yview)
250                 self.listbox.configure(yscrollcommand=scroll.set)
251                 self.listbox.pack(side="left", fill="both", expand=True)
252                 scroll.pack(side="right", fill="y")
253                 self.listbox.bind("<<ListboxSelect>>", self.on_select)
254                 self.status = tk.StringVar()
255                 ttk.Label(right, textvariable=self.status).pack(fill="x", pady=(4, 0))
256
257             # ---------------- actions ----------------
258             def _values(self):
259                 return (self.vars["name"].get(), self.vars["email"].get(), self.vars["phone"].get(),
260                         self.category.get(), self.notes.get("1.0", "end"))
261
262             def _guard(self, action, done):
263                 try:
264                     action()
265                 except ContactError as err:
266                     messagebox.showerror(type(err).__name__, str(err), parent=self.root)
267                     return False
268                 except Exception as err:                    # database down, etc.
269                     messagebox.showerror("Database error", str(err), parent=self.root)
270                     return False
271                 self.refresh()
272                 self.status.set(done)
273                 return True
274
275             def add(self):
276                 self._guard(lambda: self.repo.add(*self._values()), "Record inserted.")
277
278             def update(self):
279                 if self.selected_id is None:
280                     messagebox.showinfo("Update", "Select a contact first.", parent=self.root)
281                     return
282                 self._guard(lambda: self.repo.update(self.selected_id, *self._values()),
283                             "Record updated.")
284
285             def delete(self):
286                 if self.selected_id is None:
287                     messagebox.showinfo("Delete", "Select a contact first.", parent=self.root)
288                     return
```

```
                if messagebox.askyesno("Delete", "Delete the selected contact?", parent=self.root):
                    if self._guard(lambda: self.repo.delete(self.selected_id), "Record deleted."):
                        self.clear()

    def clear(self):
        self.selected_id = None
        for v in self.vars.values():
            v.set("")
        self.category.set("Other")
        self.notes.delete("1.0", "end")
        self.listbox.selection_clear(0, "end")

    def refresh(self):
        try:
            rows, total = self.repo.search(self.search_var.get(), self.field_var.get(),
                                           self.contains_var.get(), self.sort_var.get())
        except Exception as err:
            self.status.set(f"Search failed: {err}")
            return
        self.listbox.delete(0, "end")
        self.row_ids = []
        for cid, name, email, phone, category in rows:
            self.listbox.insert("end", f"{name[:20]:<20} {email[:28]:<28} "
                                       f"{(phone or '-'):<13} {category}")
            self.row_ids.append(cid)
        shown = f"showing first {len(rows)} of {total}" if total > len(rows) else f"{total}"
        self.status.set(f"{shown} contact(s) · sorted by {self.sort_var.get()}")

    def on_select(self, _event):
        sel = self.listbox.curselection()
        if not sel:
            return
        row = self.repo.get(self.row_ids[sel[0]])
        if not row:
            return
        self.selected_id, name, email, phone, category, notes = row
        self.vars["name"].set(name)
        self.vars["email"].set(email)
        self.vars["phone"].set(phone or "")
        self.category.set(category)
        self.notes.delete("1.0", "end")
        self.notes.insert("1.0", notes or "")

    root = tk.Tk()
    return root, ContactApp(root)


def connect(args):
    import mysql.connector
    password = args.password if args.password is not None else os.environ.get("DB_PASSWORD")
    if password is None:
        import tkinter as tk
        from tkinter import simpledialog
        tmp = tk.Tk()
        tmp.withdraw()
        password = simpledialog.askstring("MySQL", f"Password for {args.user}:", show="*") or ""
        tmp.destroy()
    return mysql.connector.connect(host=args.host, port=args.port, user=args.user,
                                   password=password, database=args.database)


def main():
    ap = argparse.ArgumentParser(description="Tkinter contact manager (MySQL)")
    ap.add_argument("--host", default="localhost")
    ap.add_argument("--port", type=int, default=3306)
    ap.add_argument("--user", default="root")
    ap.add_argument("--password")
    ap.add_argument("--database", default="college")
    args = ap.parse_args()
    try:
        import mysql.connector
    except ImportError:
```

```
361             print("ERROR: mysql-connector-python is not installed (pip install mysql-connector-python)")
362             sys.exit(1)
363         try:
364             conn = connect(args)
365         except mysql.connector.Error as err:
366             print(f"DATABASE ERROR: {getattr(err, 'msg', err)}")
367             sys.exit(1)
368         root, _app = run_gui(ContactRepository(conn))
369         root.mainloop()
370         conn.close()
371
372
373 if __name__ == "__main__":
374     main()
```

Test cases — 7 of 7 passed

#	Test case	Input	Output (expected = actual)	Result
1	**Sample: add Asha, asha@adit.edu, Faculty -> visible in list** sample	`Add name=Asha, email=asha@adit.edu, category=Faculty; search name 'as'`	`inserted id 1` `[(1, 'Asha', 'asha@adit.edu', None, 'Faculty')]`	**PASS**
2	**Duplicate email rejected (different letter case)** invalid	`add Asha K, ASHA@Adit.edu`	`DuplicateEmailError: A contact with this email already exists.`	**PASS**

3	**Email regex rejects malformed addresses**
	invalid
		`asha`
		`asha@adit`
		`a..b@adit.edu`
		`asha@adit.e`
		`asha adit@x.com`
		`@adit.edu`
			`ValidationError: 'asha' is not a valid email address.`
			`ValidationError: 'asha@adit' is not a valid email address.`
			`ValidationError: 'a..b@adit.edu' is not a valid email address.`
			`ValidationError: 'asha@adit.e' is not a valid email address.`
			`ValidationError: 'asha adit@x.com' is not a valid email address.`
			`ValidationError: '@adit.edu' is not a valid email address.`
				PASS

4	**Name, phone and category validation**
	invalid
		`name '1abc' / phone '12345' / category 'Boss'`	`ValidationError: Name must start with a letter (letters, spaces, . ' - ; max 100).`
			`ValidationError: Phone must have 10-15 digits (optional leading +).`
			`ValidationError: Category must be one of: Faculty, Student, Staff, Family, Friend, Other.`
				PASS

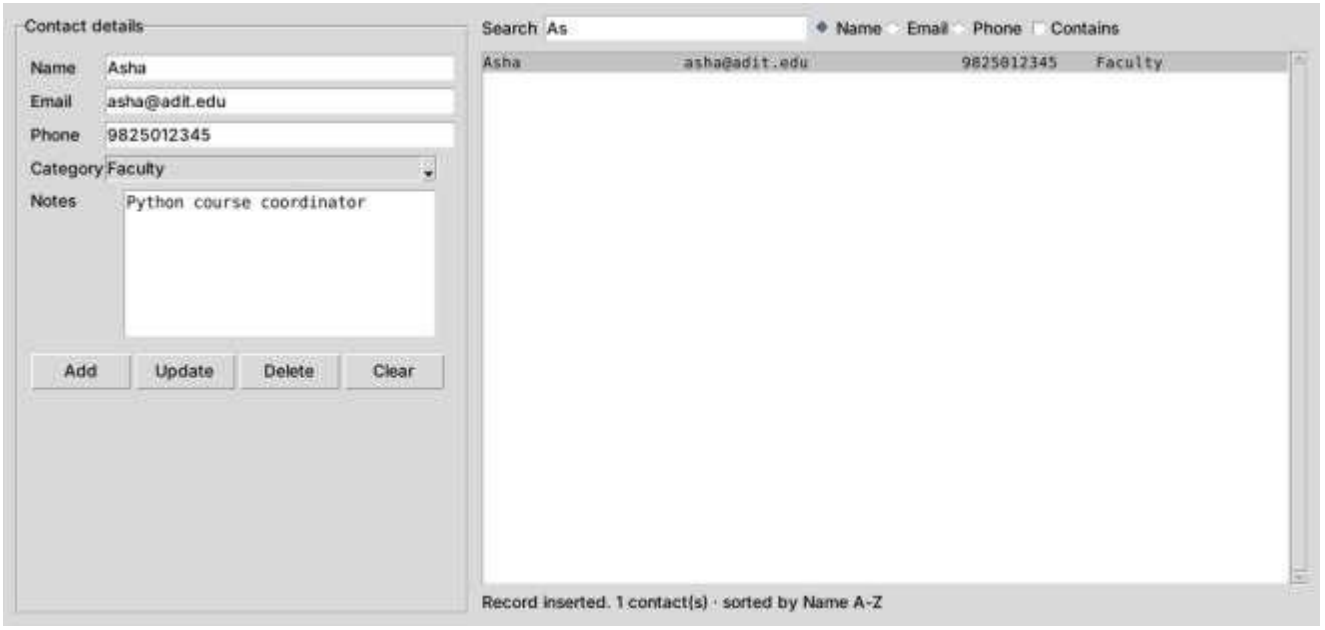
5	**Update, update to a taken email, delete, delete again**
	custom
		`add Bharat (+91 98765-43210) -> update email/category -> update to asha's email -> delete twice -> update id 9999`	`OK`
			`(3, 'Bharat Shah', 'bharat.shah@adit.edu', '9876543210', 'Staff', '')`
			`DuplicateEmailError: A contact with this email already exists.`
			`OK`
			`NotFoundError: Contact 3 does not exist.`
			`NotFoundError: Contact 99999 no longer exists.`
				PASS

6	**10,000+ contacts: prefix, contains and wildcard-escaped search**
	performance
		`10,000 generated contacts`	`prefix 'Meera': 993 matches, shown 500`
			`contains 'college3' in email: 1111 matches`
			`literal '%': 0 matches`
				PASS

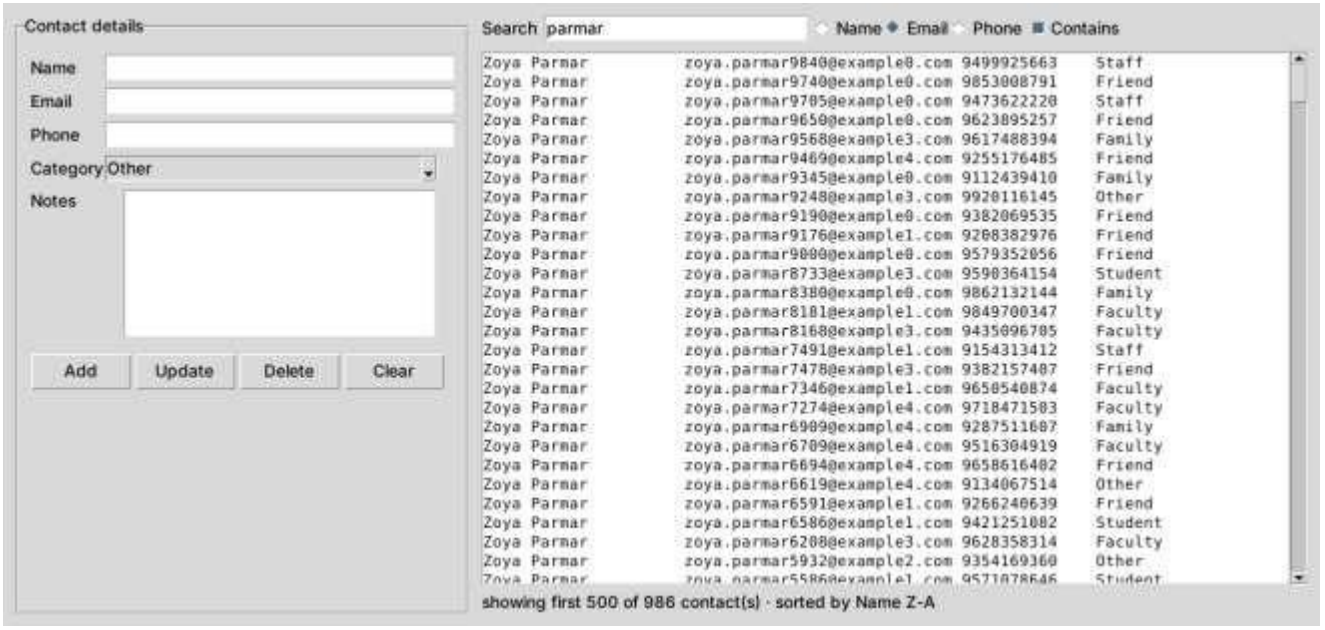
7	**EXPLAIN: prefix search uses the index, contains does a full scan** custom	`EXPLAIN on both search forms`	`prefix: type=range key=idx_contact_name` `contains: type=index key=idx_contact_name`	**PASS**

10,000+ contacts: prefix, contains and wildcard-escaped search: Measured: prefix search 9.4 ms, contains search 18.3 ms.

Screenshots



Sample action: Asha (asha@adit.edu, Faculty) added; validated, inserted and visible in the list



'Contains' email search for 'parmar' across 10,000 contacts, sorted Name Z-A from the Sort menu (first 500 of 986 shown)

Q6. Log Anomaly Detector using Sliding Window

CO-1, CO-3, CO-4 | Bloom L4 Analyze | Regular expressions, sliding window, queues, dictionaries, security analytics

Problem

From time-sorted login logs, find every user with at least X failed logins inside a T-minute window followed by a successful login from a different IP; print the users in order with the time of the first suspicious success.

Approach

- A regular expression parses each line: time (HH:MM, with optional :SS and optional date), user, an IPv4 or IPv6 address, and the status (FAIL/FAILED/FAILURE or SUCCESS/OK, any case). Impossible values such as 25:00 or 300.1.1.1 are rejected and the line is skipped.
- **Sliding window per user:** a d e q u e of (time, ip) holds only the failures still inside the window, and a C o u n t e r counts their IPs. Before each check, failures older than T minutes are popped from the left.
- On a success at time s: if at least X failures remain in [s - T, s] and the success IP is not in the Counter, the user is suspicious. Checking 'different IP' is an O(1) Counter lookup.
- A success ends the failure streak, so the window is cleared. A user who mistypes twice and then logs in normally is not flagged later.
- Every line is appended and popped at most once, so the whole run is O(n). Unsorted input is detected and reported.

Complexity

Time O(n). **Space** O(users + failures currently inside windows).

Interpretation, since the paper leaves it open: 'different IP' means an IP none of the counted failures came from. The window is inclusive (a failure exactly T minutes before the success still counts), and the tests check both edges.

How to run

```
python ENROLLMENT_2_Q6.py < sample_input.txt
```

Source code — ENROLLMENT_2_Q6.py

```python
"""
Q6. Log Anomaly Detector using Sliding Window
Course : Programming with Python (202044504) - Assignment 2
CO     : CO-1, CO-3, CO-4

A user is SUSPICIOUS when a successful login happens at time s and
    * at least X failed logins of that user lie in the window [s - T, s], and
    * the success comes from an IP address that none of those failures used.
For each suspicious user the time of the FIRST such success is printed;
users are printed in lexicographic order.

Sliding window per user
    failures[user] : deque of (minute, ip)   - only failures still inside the window
    ip_count[user] : Counter ip -> number of window failures from that ip
    New failure    : append, then drop failures older than T minutes (popleft).
    Success        : drop old failures, then
                     len(deque) >= X  and  ip not in ip_count  ->  suspicious.
                     A success ends the failure streak, so the window is cleared.
    Every log line is appended and popped at most once -> O(n) total.

Log line   : HH:MM user ip status         (HH:MM:SS and "YYYY-MM-DD HH:MM" also accepted)
             status FAIL/FAILED/FAILURE or SUCCESS/OK (any case)
Times are converted to minutes since the first day; logs must be sorted by time.
Malformed lines are skipped and counted (reported on stderr).

Run    : python ENROLLMENT_2_Q6.py < logs.txt
Time   : O(n)         Space : O(users + failures currently inside windows)
"""

import re
import sys
from collections import Counter, deque
from datetime import date

LINE_RE = re.compile(
    r"^\s*(?:(\d{4})-(\d{2})-(\d{2})[ T])?"             # optional date
    r"(\d{1,2}):(\d{2})(?::(\d{2}))?\s+"                  # time
    r"(\S+)\s+"                                          # user
    r"((?:\d{1,3}\.){3}\d{1,3}|[0-9A-Fa-f:]+:[0-9A-Fa-f:]*)\s+"    # IPv4 or IPv6
    r"(FAIL|FAILED|FAILURE|SUCCESS|OK)\s*$", re.IGNORECASE)


class InputError(Exception):
    pass


def parse_line(line):
    """Return (minutes, user, ip, is_success) or None if the line is malformed."""
    m = LINE_RE.match(line)
    if not m:
        return None
    y, mo, d, hh, mm, ss, user, ip, status = m.groups()
    hh, mm = int(hh), int(mm)
    if hh > 23 or mm > 59 or (ss and int(ss) > 59):
        return None
    if "." in ip and any(int(part) > 255 for part in ip.split(".")):
        return None
    minutes = hh * 60 + mm + (int(ss) / 60 if ss else 0)
    if y:
        try:
            minutes += date(int(y), int(mo), int(d)).toordinal() * 1440
        except ValueError:
            return None
    display = f"{y}-{mo}-{d} {hh:02d}:{mm:02d}" if y else f"{hh:02d}:{mm:02d}"
    if ss:
        display += f":{int(ss):02d}"
    return minutes, user, ip, status.upper() in ("SUCCESS", "OK"), display


def detect(events, x, t):
    """events: iterable of parsed tuples. Returns {user: display_time}."""
    failures = {}            # user -> deque[(minute, ip)]
```

```
        ip_count = {}            # user -> Counter(ip)
        found = {}
        last_time = None
        for minute, user, ip, success, display in events:
            if last_time is not None and minute < last_time:
                raise InputError(f"logs are not sorted by time (at {display})")
            last_time = minute
            if user in found:
                continue                              # only the first detection is needed
            window = failures.get(user)
            if window is None:
                window = failures[user] = deque()
                ip_count[user] = Counter()
            counts = ip_count[user]
            while window and minute - window[0][0] > t:          # slide: drop old failures
                _, old_ip = window.popleft()
                counts[old_ip] -= 1
                if not counts[old_ip]:
                    del counts[old_ip]
            if not success:
                window.append((minute, ip))
                counts[ip] += 1
                continue
            if len(window) >= x and ip not in counts:
                found[user] = display
            window.clear()                            # a success ends the failure streak
            counts.clear()
        return found


def read_input(stream, log=sys.stderr):
    """Header (X T, n), then generator over parsed log lines."""
    def next_line():
        while line := stream.readline():
            if line.strip():
                return line
        raise InputError("unexpected end of input")
    try:
        x, t = map(int, next_line().split())
        n = int(next_line())
    except ValueError:
        raise InputError("first lines must be 'X T' and 'n'") from None
    if not (1 <= x <= 100 and 1 <= t <= 1440 and 1 <= n <= 2_000_000):
        raise InputError("X, T or n outside constraints")

    def events():
        bad = 0
        for i in range(n):
            line = stream.readline()
            if not line:
                print(f"WARNING: expected {n} log lines, got {i}", file=log)
                break
            parsed = parse_line(line)
            if parsed is None:
                bad += 1
                if bad <= 5:
                    print(f"WARNING: skipped malformed line: {line.strip()!r}", file=log)
                continue
            yield parsed
        if bad > 5:
            print(f"WARNING: {bad} malformed lines skipped in total", file=log)
    return x, t, events()


def main():
    try:
        x, t, events = read_input(sys.stdin)
        found = detect(events, x, t)
    except InputError as err:
        print(f"INPUT ERROR: {err}")
        sys.exit(1)
    if not found:
```

```
            print("NO_SUSPICIOUS_USERS")
        for user in sorted(found):
            print(user, found[user])


if __name__ == "__main__":
    main()
```

Test cases — 8 of 8 passed

#	Test case	Input	Output (expected = actual)	Result
1	**Sample test** sample	3 10 5 00:00 u1 1.1.1.1 FAIL 00:03 u1 1.1.1.1 FAIL 00:05 u1 1.1.1.1 FAIL 00:07 u1 2.2.2.2 SUCCESS 00:08 u2 3.3.3.3 SUCCESS	u1 00:07	PASS
2	**Invalid: logs not sorted by time** invalid	3 10 8 00:00 a 1.1.1.1 FAIL 00:05 a 1.1.1.1 FAIL 00:10 a 1.1.1.1 FAIL 00:10 a 9.9.9.9 SUCCESS 00:00 b 1.1.1.1 FAIL	INPUT ERROR: logs are not sorted by time (at 00:00)	PASS
3	**Window edge: success at T (counts) vs T+1 (expired)** boundary	3 10 8 00:00 a 1.1.1.1 FAIL 00:00 b 1.1.1.1 FAIL 00:05 a 1.1.1.1 FAIL 00:05 b 1.1.1.1 FAIL 00:10 a 1.1.1.1 FAIL 00:10 b 1.1.1.1 FAIL 00:10 a 9.9.9.9 SUCCESS 00:11 b 9.9.9.9 SUCCESS	a 00:10	PASS
4	**Success from a failure IP is not suspicious; a success resets the streak** custom	2 30 9 10:00 ravi 5.5.5.5 FAIL 10:01 ravi 6.6.6.6 FAIL 10:02 ravi 6.6.6.6 SUCCESS 10:03 ravi 7.7.7.7 SUCCESS 10:04 neha 5.5.5.5 FAIL 10:05 neha 5.5.5.5 failed 10:06 neha 8.8.8.8 ok 10:07 neha 9.9.9.9 FAIL 10:08 neha 9.9.9.9 FAIL	neha 10:06	PASS
5	**Users sorted, first detection only, dates + seconds, malformed lines skipped** custom	1 5 7 2026-03-01 23:58:10 zed 1.1.1.1 FAIL 2026-03-02 00:01:00 zed 2.2.2.2 SUCCESS 2026-03-02 00:02:00 amy 1.1.1.1 FAIL this line is garbage 2026-03-02 00:03:00 amy 300.1.1.1 SUCCESS 2026-03-02 00:04:00 amy 3.3.3.3 SUCCESS 2026-03-02 00:09:00 zed 4.4.4.4 SUCCESS	amy 2026-03-02 00:04:00 zed 2026-03-02 00:01:00	PASS
6	**No suspicious user** boundary	5 10 2 00:00 u1 1.1.1.1 FAIL 00:01 u1 2.2.2.2 SUCCESS	NO_SUSPICIOUS_USERS	PASS
7	**Invalid: X out of range** invalid	0 10 1 00:00 u1 1.1.1.1 FAIL	INPUT ERROR: X, T or n outside constraints	PASS
8	**Max size: n = 2,000,000 lines, 20,000 users** performance	70% failures, random IPs	finished in 13.20 s, 492 suspicious users	PASS

Q7. Generator-Based JSON ETL Pipeline

CO-1, CO-2, CO-4 | Bloom L6 Create | Generators, iterators, file handling, JSON, exception handling, aggregation

Problem

An ETL pipeline built from generator functions that reads a large JSON-lines file of sensor records, filters corrupted records, transforms units, groups by device and reports count, min, max and average temperature and the corrupted count, without loading the whole file.

Approach

- Four chained generators: `read_lines` (extract), `parse_json`, `validate` (transform), and `aggregate` (load). Each pulls one record at a time from the one before, so memory does not grow with file size. The test measured a peak of about 11 MB while processing a 259 MB file.
- **Unit transform:** `temperature_f` and `temperature_k` are converted to Celsius. Numeric strings such as `"21.5"` are cleaned to numbers; text, booleans, NaN and Infinity are corrupt.
- **Validation:** device_id is required; temperature must be physically plausible (-90 to 70 °C); humidity is required and must be 0 to 100; a timestamp, if present, must be ISO-8601. A record failing any rule counts as corrupted for its device, and records with no usable device (bad JSON, missing id) are reported as UNATTRIBUTED.
- **Running statistics:** each device keeps only count, min, max, sum and corrupted count in a `__slots__` class, so space is O(devices) regardless of how many records there are.
- With `--verbose` the line number and reason for every rejected record are printed to stderr, which makes the cleaning auditable.

Complexity

Time O(number of records). **Space** O(number of devices).

> The sample record has no timestamp although the input format lists one, so timestamp is treated as optional and validated only when present.

How to run

```
python ENROLLMENT_2_Q7.py sensors.jsonl [--verbose]
```

Source code — ENROLLMENT_2_Q7.py

```python
"""
Q7. Generator-Based JSON ETL Pipeline
Course : Programming with Python (202044504) - Assignment 2
CO     : CO-1, CO-2, CO-4

Pipeline of generator functions. Each stage pulls ONE record at a time from
the previous stage, so the file is never loaded into memory:

    read_lines(path)            -> (line_no, text)            EXTRACT
    parse_json(lines)           -> (line_no, dict | error)
    validate(records)           -> Reading | Corrupted        TRANSFORM
    aggregate(readings)         -> {device: DeviceStats}       LOAD / summarise

Validation and transformation
    device_id     : required, non-empty string (max 64 characters)
    temperature   : temperature_c, or temperature_f / temperature_k which are
                    CONVERTED to Celsius (unit transform). Numeric strings such
                    as "31.5" are accepted; "bad", NaN, booleans are not.
                    Must be physically plausible: -90 .. 70 degC.
    humidity      : required, numeric, 0 .. 100 (%)
    timestamp     : optional; if present must be ISO-8601 (2026-01-01T10:00:00)
A record failing any rule is CORRUPTED and counted for its device. If even
the device is unknown (bad JSON, missing device_id) it is counted under
"unattributed".

Output per device (sorted by device_id):
    D1 count=1 min=30 max=30 avg=30.00 corrupted=1

Run   : python ENROLLMENT_2_Q7.py sensors.jsonl
Time  : O(number of records)          Space : O(number of devices)
"""

import json
import math
import sys
from datetime import datetime

TEMP_RANGE_C = (-90.0, 70.0)


class Corrupted:
    """A rejected record (device may be None) with the reason."""
    __slots__ = ("device", "reason", "line_no")

    def __init__(self, device, reason, line_no):
        self.device, self.reason, self.line_no = device, reason, line_no


class DeviceStats:
    """Running statistics: O(1) memory per device, whatever the record count."""
    __slots__ = ("count", "min", "max", "total", "corrupted")

    def __init__(self):
        self.count, self.min, self.max, self.total, self.corrupted = 0, math.inf, -math.inf, 0.0, 0

    def add(self, temp):
        self.count += 1
        self.total += temp
        self.min = min(self.min, temp)
        self.max = max(self.max, temp)


#---------------------------------------------------------------------------------------------- EXTRACT
def read_lines(path):
    with open(path, encoding="utf-8", errors="replace") as fh:
        for line_no, line in enumerate(fh, start=1):
            if line.strip():
                yield line_no, line


def parse_json(lines):
    for line_no, text in lines:
```

```
            try:
                yield line_no, json.loads(text)
            except json.JSONDecodeError as err:
                yield line_no, Corrupted(None, f"invalid JSON ({err.msg})", line_no)
77
78
#-------------------------------------------------------------------------------------------------TRANSFORM
def to_number(value, field):
    """Accept int/float or a numeric string; reject bool, NaN, inf, text."""
    if isinstance(value, bool) or value is None:
        raise ValueError(f"{field} missing or not numeric")
    if isinstance(value, str):
        try:
            value = float(value.strip())
        except ValueError:
            raise ValueError(f"{field} '{value}' is not numeric") from None
    if not isinstance(value, (int, float)) or not math.isfinite(value):
        raise ValueError(f"{field} is not a finite number")
    return float(value)
92
93
def temperature_c(rec):
    """Unit transform: Fahrenheit and Kelvin are converted to Celsius."""
    if "temperature_c" in rec:
        return to_number(rec["temperature_c"], "temperature_c")
    if "temperature_f" in rec:
        return (to_number(rec["temperature_f"], "temperature_f") - 32) * 5 /  9
    if "temperature_k" in rec:
        return to_number(rec["temperature_k"], "temperature_k") - 273.15
    raise ValueError("temperature missing")
103
104
def validate(records):
    """Yield (device, temp_c) for good records, Corrupted for bad ones."""
    for line_no, rec in records:
        if isinstance(rec, Corrupted):
            yield rec
            continue
        if not isinstance(rec, dict):
            yield Corrupted(None, "line is not a JSON object", line_no)
            continue
        device = rec.get("device_id")
        if not isinstance(device, str) or not device.strip() or len(device) >  64:
            yield Corrupted(None, "device_id missing or invalid", line_no)
            continue
        device = device.strip()
        try:
            temp = temperature_c(rec)
            if not TEMP_RANGE_C[0] <= temp <= TEMP_RANGE_C[1]:
                raise ValueError(f"temperature {temp:.1f} C outside plausible range")
            humidity = to_number(rec.get("humidity"), "humidity")
            if not 0 <= humidity <= 100:
                raise ValueError(f"humidity {humidity} outside 0-100")
            if "timestamp" in rec:
                datetime.fromisoformat(str(rec["timestamp"]))
        except ValueError as err:
            yield Corrupted(device, str(err), line_no)
            continue
        yield device, temp
132
133
#-------------------------------------------------------------------------------------------------LOAD
def aggregate(items, log=None):
    stats = {}
    unattributed = 0
    for item in items:
        if isinstance(item, Corrupted):
            if item.device is None:
                unattributed += 1
            else:
                stats.setdefault(item.device, DeviceStats()).corrupted +=  1
            if log is not None:
```

```
                print(f"line {item.line_no}: {item.reason}", file=log)
            continue
        device, temp = item
        stats.setdefault(device, DeviceStats()).add(temp)
    return stats, unattributed


def fmt(x):
    return f"{x:.2f}".rstrip("0").rstrip(".") if x != int(x) else str(int(x))


def report(stats, unattributed):
    lines = []
    for device in sorted(stats):
        s = stats[device]
        if s.count:
            lines.append(f"{device} count={s.count} min={fmt(s.min)} max={fmt(s.max)} "
                         f"avg={s.total / s.count:.2f} corrupted={s.corrupted}")
        else:
            lines.append(f"{device} count=0 min=NA max=NA avg=NA corrupted={s.corrupted}")
    if unattributed:
        lines.append(f"UNATTRIBUTED corrupted={unattributed}")
    return "\n".join(lines) if lines else "NO_RECORDS"


def main():
    args = [a for a in sys.argv[1:] if a != "--verbose"]
    if len(args) != 1:
        print("Usage: python ENROLLMENT_2_Q7.py <file.jsonl> [--verbose]")
        sys.exit(1)
    log = sys.stderr if "--verbose" in sys.argv else None
    try:
        pipeline = validate(parse_json(read_lines(args[0])))
        print(report(*aggregate(pipeline, log)))
    except FileNotFoundError:
        print(f"ERROR: file not found: {args[0]}")
        sys.exit(1)
    except (IsADirectoryError, PermissionError) as err:
        print(f"ERROR: cannot read file: {err}")
        sys.exit(1)


if __name__ == "__main__":
    main()
```

Test cases — 5 of 5 passed

#	Test case	Input	Output (expected = actual)	Result
1	**Sample test** sample	sensors.jsonl: {"device_id":"D1","temperature_c":30,"humidity":50} {"device_id":"D1","temperature_c":"bad","humidity":45}	D1 count=1 min=30 max=30 avg=30.00 corrupted=1	PASS
2	**Unit conversion F/K->C, numeric strings, 6 kinds of corruption** custom	m.jsonl: {"device_id":"T9","timestamp":"2026-01-01T10:00:00","temperature_f":98.6,"humidity":40} {"device_id":"T9","temperature_k":300.15,"humidity":"55.5"} {"device_id":"T9","temperature_c":"21.5","humidity":61} {"device_id":"A1","temperature_c":NaN,"humidity":40} {"device_id":"A1","temperature_c":25,"humidity":120} {"device_id":"A1","temperature_c":25} {"device_id":"A1","temperature_c":true,"humidity":30} {"device_id":"A1","timestamp":"yesterday","temperature_c":22,"humidity":30} {"device_id":"A1","temperature_c":-120,"humidity":30} {"temperature_c":20,"humidity":30} {this is not json [1, 2, 3]	A1 count=0 min=NA max=NA avg=NA corrupted=6 T9 count=3 min=21.5 max=37 avg=28.50 corrupted=0 UNATTRIBUTED corrupted=3	PASS

#	Test case	Input	Output (expected = actual)	Result
3	**Boundary: empty file** boundary	`e.jsonl (blank)`	`NO_RECORDS`	**PASS**
4	**Invalid: missing file** invalid	`nofile.jsonl`	`ERROR: file not found: nofile.jsonl`	**PASS**
5	**Large: 3,000,000 records (259 MB), 1000 devices** performance	`every 97th record corrupted`	`finished in 18.86 s, peak memory 11 MB`	**PASS**

Q8. Matrix Path Optimizer with Dynamic Programming

CO-1, CO-2 | Bloom L5 Evaluate | Dynamic programming, matrix algorithms, path counting, optimisation

Problem

Find the maximum-score path from the top-left to the bottom-right of an n x m matrix moving right, down or diagonally down-right, avoiding blocked cells (X); also print the number of maximum-score paths modulo 1,000,000,007, or IMPOSSIBLE.

Approach

- **Recurrence:** best(i, j) = cell(i, j) + max(best(up), best(left), best(diagonal)); ways(i, j) = the sum of ways over the neighbours that reach that maximum, modulo 10^9+7. Blocked or unreachable cells have no score and 0 ways.
- Ties are handled while the maximum is chosen: an equal predecessor adds its ways, and a better one replaces them. This counts every distinct optimal path exactly once.
- **O(m) space:** row i only needs row i-1, so two rows are kept and the matrix itself is read one row at a time, never stored whole.
- Scores stay exact Python integers (at most about 4×10^9 in magnitude), and a sentinel (None) marks unreachable cells, so negative scores are handled correctly.
- Rows may be space-separated (needed for negative numbers) or, as in the paper's sample, one token of m single-digit cells such as 4 X 6 .

Complexity

Time O(n·m): constant work per cell. **Space** O(m) for the two rows.

The 2000 × 2000 all-zero test is checked against an independent formula: with these three moves, the number of paths is the Delannoy number D(1999, 1999), computed separately in the test script.

How to run

```
python ENROLLMENT_2_Q8.py < sample_input.txt
```

Source code — ENROLLMENT_2_Q8.py

```python
"""
Q8. Matrix Path Optimizer with Dynamic Programming
Course : Programming with Python (202044504) - Assignment 2
CO     : CO-1, CO-2

Moves: right, down, diagonal down-right. X = blocked cell.
For every cell (i, j):
    best[i][j]  = cell[i][j] + max(best of up, left, up-left)
    ways[i][j]  = sum of ways of those neighbours whose best equals that max
                  (mod 1 000 000 007)
Blocked or unreachable cells have best = NONE and ways = 0.
Answer: best and ways of the bottom-right cell, or IMPOSSIBLE.

Space optimisation: row i only needs row i-1, so two rows of length m are
kept (prev and cur) instead of the full n x m table -> O(m) memory, and the
matrix itself is read one row at a time from the input.

Input rows: m values separated by spaces (negative numbers allowed), or - as
in the sample - one token of exactly m single-digit cells, e.g. "4X6".

Run   : python ENROLLMENT_2_Q8.py < input.txt
Time  : O(n*m)          Space : O(m)
"""

import sys

MOD = 1_000_000_007
NONE = None             # marks a blocked / unreachable cell
LIMIT = 10 ** 6


class InputError(Exception):
    pass


def parse_row(line, m, row_no):
    """Return list of int scores, with None for blocked cells."""
    tokens = line.split()
    if len(tokens) == 1 and m > 1 and len(tokens[0]) == m:
        tokens = list(tokens[0])                    # compact form: "4X6"
    if len(tokens) != m:
        raise InputError(f"row {row_no}: expected {m} values, got {len(tokens)}")
    row = []
    for tok in tokens:
        if tok in ("X", "x"):
            row.append(NONE)
            continue
        try:
            value = int(tok)
        except ValueError:
            raise InputError(f"row {row_no}: '{tok}' is neither an integer nor X") from None
        if not -LIMIT <= value <= LIMIT:
            raise InputError(f"row {row_no}: value {value} outside [-10^6, 10^6]")
        row.append(value)
    return row


def best_path(rows, n, m):
    """rows: iterator over parsed rows. Returns (score, count) or None."""
    prev_best = [NONE] * m
    prev_ways = [0] * m
    for i in range(n):
        row = next(rows)
        cur_best = [NONE] * m
        cur_ways = [0] * m
        for j in range(m):
            cell = row[j]
            if cell is NONE:
                continue
            # predecessor 1: up
            best, ways = prev_best[j], prev_ways[j]
            if j:
```

```
                # predecessor 2: left
                b = cur_best[j - 1]
                if b is not NONE:
                    if best is NONE or b > best:
                        best, ways = b, cur_ways[j - 1]
                    elif b == best:
                        ways += cur_ways[j - 1]
                # predecessor 3: diagonal up-left
                b = prev_best[j - 1]
                if b is not NONE:
                    if best is NONE or b > best:
                        best, ways = b, prev_ways[j - 1]
                    elif b == best:
                        ways += prev_ways[j - 1]
            if best is NONE:
                if i == 0 and j == 0:           # the start cell
                    cur_best[0], cur_ways[0] = cell, 1
                continue                        # unreachable cell
            cur_best[j] = best + cell
            cur_ways[j] = ways % MOD
        prev_best, prev_ways = cur_best, cur_ways
    if prev_best[m - 1] is NONE:
        return None
    return prev_best[m - 1], prev_ways[m - 1]


def solve(stream):
    header = stream.readline().split()
    if len(header) != 2:
        raise InputError("first line must be: n m")
    try:
        n, m = int(header[0]), int(header[1])
    except ValueError:
        raise InputError("n and m must be integers") from None
    if not (1 <= n <= 2000 and 1 <= m <= 2000):
        raise InputError("n and m must be between 1 and 2000")

    def rows():
        for i in range(1, n + 1):
            line = stream.readline()
            while line and not line.strip():
                line = stream.readline()
            if not line:
                raise InputError(f"expected {n} rows, got {i - 1}")
            yield parse_row(line, m, i)

    result = best_path(rows(), n, m)
    return "IMPOSSIBLE" if result is None else f"{result[0]} {result[1]}"


def main():
    try:
        print(solve(sys.stdin))
    except InputError as err:
        print(f"INPUT ERROR: {err}")
        sys.exit(1)


if __name__ == "__main__":
    main()
```

Test cases — 11 of 11 passed

#	Test case	Input	Output (expected = actual)	Result
1	**Sample test** sample	3 3 1 2 3 4 X 6 7 8 9	29 1	PASS
2	**All zeros 2x2: right-down, down-right, diagonal tie** custom	2 2 0 0 0 0	0 3	PASS

#	Test case	Input	Output (expected = actual)	Result

3	**Negative scores: least negative path wins**
	custom
		`3 3`
		`-1 -5 -1`
		`-1 -9 -1`
		`-5 -1 -1`
			`-4 1`	**PASS**

4	**Boundary: 1x1 matrix** boundary	`1 1` `7`	`7 1`	**PASS**

5	**Boundary: start blocked**
	boundary
		`2 2`
		`X 1`
		`1 1`
			`IMPOSSIBLE`	**PASS**

6	**Diagonal moves pass a broken wall**
	custom
		`3 3`
		`1 1 X`
		`1 X 1`
		`X 1 1`
			`4 2`	**PASS**

7	**A full row of X blocks every path**
	custom
		`3 3`
		`1 1 1`
		`X X X`
		`1 1 1`
			`IMPOSSIBLE`	**PASS**

8	**Invalid: wrong row length**
	invalid
		`2 3`
		`1 2 3`
		`4 5`
			`INPUT ERROR: row 2: expected 3 values, got 2`	**PASS**

9	**Invalid: value above 10^6** invalid	`1 2` `1 2000000`	`INPUT ERROR: row 1: value 2000000 outside [-10^6, 10^6]`	**PASS**
10	**Max size 2000x2000 zeros: count = Delannoy number mod 1e9+7** performance	`4,000,000 cells`	`finished in 2.40 s, count matches formula (188584237)`	**PASS**
11	**Max size 2000x2000 random values, 10% blocked** performance	`4,000,000 cells`	`finished in 1.93 s`	**PASS**

Q9. OOP Inventory Merger with Operator Overloading

CO-1, CO-2, CO-4 | Bloom L6 Create | OOP, operator overloading, dictionaries, file handling, data modelling

Problem

Design Product and Inventory classes supporting add, delete, update stock, comparison by total value and merging with operator overloading. When merging, the same product id adds stock, keeps the lower purchase price and keeps the higher selling price.

Approach

- **Product:** validated properties (stock is an integer from 0 to 2^{63}-1; prices are non-negative Decimals, so 30.50 stays exact). `p1 + p2` merges two records of the same product and raises an error for different ids.
- **Inventory operators:** `a + b` returns a new merged inventory without changing a or b (shown in the test by reporting A after MERGE); `a += b` merges in place; `<` and `==` compare total value, and `@total_ordering` supplies the other comparisons; `len()`, `in`, `[]` and iteration also work.
- Products are stored in a dictionary by id, so add, delete, update and each merge step are O(1) on average, and merging is O(n + m).
- Custom exceptions (DuplicateProductError, ProductNotFoundError, InvalidValueError) are caught per operation, so an error line is printed and the remaining operations still run. Bad lines in inventory files are skipped with a warning.
- UPDATE accepts an absolute value (`20`) or a relative change (`+5`, `-3`); stock can never become negative.

Complexity

Time O(n + m + q) on average with dictionaries (printing a sorted report adds a sort). **Space** O(n + m).

Total valuation is stock × purchase price (inventory at cost), which the paper does not define. When the same id has different names in A and B, the name from A is kept.

How to run

```
python ENROLLMENT_2_Q9.py inventoryA.txt inventoryB.txt < operations.txt
```

Source code — ENROLLMENT_2_Q9.py

```python
"""
Q9. OOP Inventory Merger with Operator Overloading
Course : Programming with Python (202044504) - Assignment 2
CO     : CO-1, CO-2, CO-4

Classes
    Product     id, name, stock, purchase price, selling price (validated properties)
                p1 + p2      -> merged product (same id only)
    Inventory   dict id -> Product
                a + b        -> NEW merged inventory (a and b are not changed)
                a += b       -> merge b into a
                a < b, a == b ... -> compare by total value (functools.total_ordering)
                len(a), "P1" in a, a["P1"], iteration
    Custom exceptions: InventoryError > DuplicateProductError, ProductNotFoundError,
                       InvalidValueError

Merge rule for the same product id: stock is added, the LOWER purchase price
and the HIGHER selling price are kept. The name from the left inventory wins.
Total value of an inventory = sum(stock x purchase price)   (stock at cost).

Inventory file: one product per line ->  id name stock purchase selling
                (name may contain spaces; blank lines and # comments ignored)
Operations (standard input: first line q, then q lines)
    ADD A|B id name stock purchase selling
    DELETE A|B id
    UPDATE A|B id stock          (absolute)   or  +n / -n  (relative change)
    MERGE                        M = A + B, prints merged summary + valuation
    COMPARE                      compares A and B by total value
    REPORT A|B|M

Run   : python ENROLLMENT_2_Q9.py inventoryA.txt inventoryB.txt < operations.txt
Time  : O(n + m + q) average with dictionaries (+ sorting only when printing)
Space : O(n + m)
"""

import sys
from decimal import Decimal, InvalidOperation
from functools import total_ordering

INT64_MAX = 2 ** 63 - 1


#---------------------------------------------------------------------------------------------- exceptions
class InventoryError(Exception):
    pass


class DuplicateProductError(InventoryError):
    pass


class ProductNotFoundError(InventoryError):
    pass


class InvalidValueError(InventoryError):
    pass


def fmt(value):
    """Decimal('5.00') -> '5', Decimal('4.50') -> '4.5'."""
    text = format(Decimal(value).normalize(), "f")
    return text


#---------------------------------------------------------------------------------------------- Product
class Product:
    __slots__ = ("_pid", "_name", "_stock", "_purchase", "_selling")

    def __init__(self, pid, name, stock, purchase, selling):
        if not pid or any(c.isspace() for c in pid):
            raise InvalidValueError(f"invalid product id '{pid}'")
```

```
            if not name.strip():
                raise InvalidValueError(f"product {pid} has no name")
            self._pid = pid
            self._name = " ".join(name.split())
            self.stock = stock              # property setters validate
            self._purchase = self._price(purchase, "purchase price")
            self._selling = self._price(selling, "selling price")

        @staticmethod
        def _price(value, label):
            try:
                price = Decimal(str(value))
            except InvalidOperation:
                raise InvalidValueError(f"{label} '{value}' is not a number") from None
            if not price.is_finite() or price < 0 or price > INT64_MAX:
                raise InvalidValueError(f"{label} {value} out of range")
            return price

        # read-only identity, validated stock
        pid = property(lambda self: self._pid)
        name = property(lambda self: self._name)
        purchase = property(lambda self: self._purchase)
        selling = property(lambda self: self._selling)

        @property
        def stock(self):
            return self._stock

        @stock.setter
        def stock(self, value):
            try:
                value = int(value)
            except (TypeError, ValueError):
                raise InvalidValueError(f"stock '{value}' is not an integer") from None
            if not 0 <= value <= INT64_MAX:
                raise InvalidValueError(f"stock {value} must be between 0 and 2^63-1")
            self._stock = value

        @property
        def value(self):
            return self._stock * self._purchase

        def copy(self):
            return Product(self._pid, self._name, self._stock, self._purchase, self._selling)

        def __add__(self, other):
            """Merge two records of the SAME product."""
            if not isinstance(other, Product):
                return NotImplemented
            if other.pid != self.pid:
                raise InventoryError(f"cannot merge different products {self.pid} and {other.pid}")
            return Product(self.pid, self.name, self.stock + other.stock,
                           min(self.purchase, other.purchase), max(self.selling, other.selling))

        def __repr__(self):
            return (f"{self.pid} {self.name} stock={self.stock} "
                    f"purchase={fmt(self.purchase)} selling={fmt(self.selling)}")


    #-------------------------------------------------------------------------------------------- Inventory
    @total_ordering
    class Inventory:
        def __init__(self, label="", products=()):
            self.label = label
            self._items = {}
            for p in products:
                self.add(p)

        # ---- container protocol ----
        def __len__(self):
            return len(self._items)

```

```
    def __contains__(self, pid):
        return pid in self._items

    def __getitem__(self, pid):
        try:
            return self._items[pid]
        except KeyError:
            raise ProductNotFoundError(f"product {pid} not found in {self.label}") from None

    def __iter__(self):
        return iter(self._items.values())

    # ---- operations ----
    def add(self, product):
        if product.pid in self._items:
            raise DuplicateProductError(f"product {product.pid} already exists in {self.label}")
        self._items[product.pid] = product

    def delete(self, pid):
        self[pid]                                   # raises if missing
        del self._items[pid]

    def update_stock(self, pid, text):
        product = self[pid]
        try:
            amount = int(text)
        except ValueError:
            raise InvalidValueError(f"stock '{text}' is not an integer") from None
        product.stock = product.stock + amount if text[0] in "+-" else amount

    @property
    def total_value(self):
        return sum((p.value for p in self._items.values()), Decimal(0))

    # ---- operator overloading ----
    def _merge_into(self, other):
        for p in other:
            mine = self._items.get(p.pid)
            self._items[p.pid] = p.copy() if mine is None else mine + p

    def __add__(self, other):
        if not isinstance(other, Inventory):
            return NotImplemented
        merged = Inventory(f"{self.label}+{other.label}", (p.copy() for p in self))
        merged._merge_into(other)
        return merged

    def __iadd__(self, other):
        if not isinstance(other, Inventory):
            return NotImplemented
        self._merge_into(other)
        return self

    def __eq__(self, other):
        if not isinstance(other, Inventory):
            return NotImplemented
        return self.total_value == other.total_value

    def __lt__(self, other):
        if not isinstance(other, Inventory):
            return NotImplemented
        return self.total_value < other.total_value

    __hash__ = None                                 # mutable -> unhashable

    def report(self):
        lines = [repr(self._items[k]) for k in sorted(self._items)]
        lines.append(f"TOTAL_VALUATION {fmt(self.total_value)}")
        return "\n".join(lines)


# ------------------------------------------------------------ file + driver
```

```
    def product_from_tokens(tokens):
        if len(tokens) < 5:
            raise InvalidValueError("expected: id name stock purchase selling")
        return Product(tokens[0], " ".join(tokens[1:-3]), *tokens[-3:])


def load_inventory(path, label, log=sys.stderr):
    inv = Inventory(label)
    with open(path, encoding="utf-8") as fh:
        for line_no, line in enumerate(fh, start=1):
            line = line.split("#", 1)[0].strip()
            if not line:
                continue
            try:
                inv.add(product_from_tokens(line.split()))
            except InventoryError as err:
                print(f"WARNING: {path} line {line_no} skipped: {err}", file=log)
    return inv


def execute(op_line, invs):
    """Run one operation; returns text to print."""
    parts = op_line.split()
    cmd = parts[0].upper()
    if cmd == "MERGE":
        invs["M"] = invs["A"] + invs["B"]
        return f"MERGE OK ({len(invs['M'])} products)\n" + invs["M"].report()
    if cmd == "COMPARE":
        a, b = invs["A"], invs["B"]
        sign = ">" if a > b else "<" if a < b else "="
        return f"COMPARE A {sign} B ({fmt(a.total_value)} vs {fmt(b.total_value)})"
    if len(parts) < 2 or parts[1].upper() not in invs:
        raise InventoryError(f"{cmd}: unknown or missing inventory name")
    inv = invs[parts[1].upper()]
    if cmd == "REPORT" and len(parts) == 2:
        return inv.report()
    if cmd == "ADD":
        product = product_from_tokens(parts[2:])
        inv.add(product)
        return f"ADD OK {product.pid} -> {inv.label}"
    if cmd == "DELETE" and len(parts) == 3:
        inv.delete(parts[2])
        return f"DELETE OK {parts[2]} <- {inv.label}"
    if cmd == "UPDATE" and len(parts) == 4:
        inv.update_stock(parts[2], parts[3])
        return f"UPDATE OK {parts[2]} stock={inv[parts[2]].stock}"
    raise InventoryError(f"cannot understand operation '{op_line}'")


def run(path_a, path_b, ops_text, log=sys.stderr):
    invs = {"A": load_inventory(path_a, "A", log), "B": load_inventory(path_b, "B", log)}
    lines = [ln.strip() for ln in ops_text.splitlines() if ln.strip()]
    if not lines:
        raise InventoryError("no operations given")
    try:
        q = int(lines[0])
    except ValueError:
        raise InventoryError("first operation line must be q") from None
    if not 0 <= q <= 300000:
        raise InventoryError("q must be between 0 and 300000")
    out = []
    for op in lines[1:q + 1]:
        try:
            out.append(execute(op, invs))
        except InventoryError as err:
            out.append(f"ERROR {type(err).__name__}: {err}")
    return "\n".join(out)


def main():
    if len(sys.argv) != 3:
        print("Usage: python ENROLLMENT_2_Q9.py inventoryA.txt inventoryB.txt < operations.txt")
```

```
            sys.exit(1)
        try:
            print(run(sys.argv[1], sys.argv[2], sys.stdin.read()))
        except FileNotFoundError as err:
            print(f"ERROR: file not found: {err.filename}")
            sys.exit(1)
        except InventoryError as err:
            print(f"INPUT ERROR: {err}")
            sys.exit(1)


if __name__ == "__main__":
    main()
```

Test cases — 5 of 5 passed

#	Test case	Input	Output (expected = actual)	Result

1	**Sample test**
	sample
		`Inventory A: P1 Pen 10 5 8`
		`Inventory B: P1 Pen 7 4 9`
		`Operation: MERGE`
			`MERGE OK (1 products)`
			`P1 Pen stock=17 purchase=4 selling=9`
			`TOTAL_VALUATION 68`
				PASS

2	**ADD / UPDATE (relative) / DELETE / MERGE / COMPARE, names with spaces**
	custom
		`A: P1 Pen 10 5 8 | P2 Blue Notebook 20 30.50 45 | P3 Ruler 5 10 12`
		`B: P2 Blue Notebook 5 28 50 | P4 Eraser 100 2 3.5`
		`8`
		`COMPARE`
		`ADD A P5 Geometry Box 3 120 150`
		`UPDATE A P1 +5`
		`UPDATE B P4 -40`
		`DELETE A P3`
		`MERGE`
		`REPORTA`
		`COMPARE`
			`COMPARE A > B (710 vs 340)`
			`ADD OK P5 -> A`
			`UPDATE OK P1 stock=15`
			`UPDATE OK P4 stock=60`
			`DELETE OK P3 <- A`
			`MERGE OK (4 products)`
			`P1 Pen stock=15 purchase=5 selling=8`
			`P2 Blue Notebook stock=25 purchase=28 selling=50`
			`P4 Eraser stock=60 purchase=2 selling=3.5`
			`P5 Geometry Box stock=3 purchase=120 selling=150`
			`TOTAL_VALUATION 1255`
			`P1 Pen stock=15 purchase=5 selling=8`
			`P2 Blue Notebook stock=20 purchase=30.5 selling=45`
			`P5 Geometry Box stock=3 purchase=120 selling=150`
			`TOTAL_VALUATION 1045`
			`COMPARE A > B (1045 vs 260)`
				PASS

3	**Errors are reported and processing continues**
	invalid
		`A as above`
		`7`
		`ADD A P1 Pencil 1 1 1`
		`DELETE B P9`
		`UPDATE A P1 -11`
		`UPDATE A P1 ten`
		`REPORT M`
		`SELL A P1 2`
		`ADD A P7 Glue -1 4 5`
			`ERROR DuplicateProductError: product P1 already exists in A`
			`ERROR ProductNotFoundError: product P9 not found in B`
			`ERROR InvalidValueError: stock -1 must be between 0 and 2^63-1`
			`ERROR InvalidValueError: stock 'ten' is not an integer`
			`ERROR InventoryError: REPORT: unknown or missing inventory name`
			`ERROR InventoryError: cannot understand operation 'SELL A P1 2'`
			`ERROR InvalidValueError: stock -1 must be between 0 and 2^63-1`
				PASS

4	**Boundary: empty inventory B; duplicate/invalid lines in a file skipped**
	boundary
		`A: P1 Pen 10 5 8 / P1 Pen 2 2 2 / P2 Clip x 1 1 / P3 OnlyName`
		`B: (empty)`
		`2`
		`MERGE`
		`COMPARE`
			`MERGE OK (1 products)`
			`P1 Pen stock=10 purchase=5 selling=8`
			`TOTAL_VALUATION 50`
			`COMPARE A > B (50 vs 0)`
			`[stderr]`
			`WARNING: /tmp/tmpn7lm2td5/bad.txt line 2 skipped: product P1 already exists in A`
			`WARNING: /tmp/tmpn7lm2td5/bad.txt line 3 skipped: stock 'x' is not an integer`
			`WARNING: /tmp/tmpn7lm2td5/bad.txt line 4 skipped: expected: id name stock purchase selling`
				PASS

5	**Max size: 2 x 500,000 products (250,000 shared), 300,000 operations** performance	`random UPDATE/ADD, then MERGE + COMPARE`	`finished in 14.75 s`	PASS

Q10. Python Data Visualization Dashboard Exporter

CO-1, CO-4 | Bloom L6 Create | Data manipulation, data visualisation, CSV, statistics, reporting

Problem

Read a CSV of student marks, clean missing values, compute subject-wise statistics and export a cleaned CSV, a summary CSV and three charts: grade distribution, subject-wise average and top performers.

Approach

- **Cleaning:** a mark that is blank, NA, AB, non-numeric or outside 0 to 100 counts as missing and is filled with that subject's mean. The new `filled_marks` column records which marks were filled, so every change can be traced. Rows with no enrollment, a duplicate enrollment, no name or no valid mark are dropped and counted.
- **Streaming in two passes** (the streaming alternative the paper allows): pass 1 accumulates count, sum, sum of squares, min, max and pass count per subject; pass 2 fills gaps with those means and writes each cleaned row immediately. Statistics use O(s) memory.
- **Top performers** come from a size-10 min-heap, O(n log 10), so the 1,000,000 students are never sorted. Grades O, A+, A, B+, B, C and F come from each student's average.
- **summary.csv** holds, per subject: valid marks, filled count, mean, standard deviation, min, max, pass count and pass rate.
- **Charts** are drawn with matplotlib's Agg backend, so no display is needed: one hue, value labels on the bars, a light grid and no chart junk, which prints well in grayscale. They are saved as 150-dpi PNG files.

Complexity

Time O(n·s). **Space** O(s) for statistics plus O(n) for the set of seen enrollments (used for duplicate detection); the data itself is streamed.

Uses matplotlib (`pip install matplotlib`), as the question allows. pandas is not needed, because the csv module streams the file in O(s) memory.

How to run

```
python ENROLLMENT_2_Q10.py marks.csv output
```

Source code — ENROLLMENT_2_Q10.py

```python
"""
Q10. Python Data Visualization Dashboard Exporter
Course : Programming with Python (202044504) - Assignment 2
CO     : CO-1, CO-4

Input : marks CSV  enrollment,name,<subject 1>,...,<subject s>    (s <= 20)
Output: cleaned_marks.csv, summary.csv, grade_distribution.png,
        subject_average.png, top_performers.png    (in the output folder)

Cleaning rules
  * A mark that is blank, "NA", "AB", non-numeric or outside 0-100 is MISSING.
  * Missing marks are filled with that subject's mean of valid marks
    (mean imputation, rounded to 2 decimals); the cleaned file has a
    filled_marks column so every imputed value stays traceable.
  * Rows with no enrollment, a duplicate enrollment, no name, or no valid
    mark at all are dropped (counted in the console report).

Grades from the student's average:  O >= 90, A+ >= 80, A >= 70, B+ >= 60,
B >= 50, C >= 40, F < 40.

Streaming design (two passes over the file, never loading it whole)
  Pass 1: validate rows, accumulate per-subject count / sum / sum of squares /
          min / max  ->  O(s) memory for the statistics.
  Pass 2: fill missing marks with the means from pass 1, write the cleaned
          row immediately, update grade counts and a size-10 heap of the top
          performers.
  Only the set of seen enrollments (for duplicate detection) grows with n.

Libraries: matplotlib (charts only) - pip install matplotlib
Run    : python ENROLLMENT_2_Q10.py marks.csv output_folder
Time   : O(n*s)          Space : O(s) statistics + O(n) enrollment set
"""

import csv
import heapq
import math
import os
import sys

TOP_K = 10
MAX_SUBJECTS = 20
GRADES = [("O", 90), ("A+", 80), ("A", 70), ("B+", 60), ("B", 50), ("C", 40), ("F", 0)]
PASS_MARK = 40

# chart styling (validated reference palette: one hue, recessive ink)
BAR = "#2a78d6"
INK = "#0b0b0b"
INK_2 = "#52514e"
GRID = "#e4e3df"


class InputError(Exception):
    pass


def grade_of(avg):
    for grade, cutoff in GRADES:
        if avg >= cutoff:
            return grade
    return "F"


def parse_mark(text):
    """Return a float mark in 0..100, or None if missing/invalid."""
    text = (text or "").strip()
    if not text:
        return None
    try:
        value = float(text)
    except ValueError:
        return None                              # NA, AB, "abc" ...
    if not math.isfinite(value) or not 0 <= value <= 100:
```

```
            return None
        return value


class SubjectStats:
    __slots__ = ("count", "total", "squares", "low", "high", "passed", "filled")

    def __init__(self):
        self.count = self.passed = self.filled = 0
        self.total = self.squares = 0.0
        self.low, self.high = math.inf, -math.inf

    def add(self, mark):
        self.count += 1
        self.total += mark
        self.squares += mark * mark
        self.low = min(self.low, mark)
        self.high = max(self.high, mark)
        self.passed += mark >= PASS_MARK

    @property
    def mean(self):
        return self.total / self.count if self.count else 0.0

    @property
    def std(self):
        if not self.count:
            return 0.0
        return math.sqrt(max(self.squares / self.count - self.mean ** 2, 0.0))


def read_header(reader):
    header = next(reader, None)
    if not header:
        raise InputError("file is empty")
    header = [h.strip() for h in header]
    if len(header) < 3 or header[0].lower() != "enrollment" or header[1].lower() != "name":
        raise InputError("header must be: enrollment,name,<subject>,...")
    subjects = header[2:]
    if len(subjects) > MAX_SUBJECTS:
        raise InputError(f"at most {MAX_SUBJECTS} subjects are supported")
    if len(set(s.lower() for s in subjects)) != len(subjects) or not all(subjects):
        raise InputError("subject names must be non-empty and unique")
    return header, subjects


def valid_rows(path):
    """Generator shared by both passes: yields (enrollment, name, marks) for
    rows that are kept, plus the reason for each dropped row."""
    seen = set()
    with open(path, newline="", encoding="utf-8-sig") as fh:
        reader = csv.reader(fh)
        _, subjects = read_header(reader)
        s = len(subjects)
        for row in reader:
            if not any(cell.strip() for cell in row):
                continue
            row = (row + [""] * (s + 2))[:s + 2]           # pad short rows
            enrollment, name = row[0].strip(), " ".join(row[1].split())
            if not enrollment:
                yield "drop", "missing enrollment"
                continue
            if enrollment in seen:
                yield "drop", "duplicate enrollment"
                continue
            seen.add(enrollment)
            if not name:
                yield "drop", "missing name"
                continue
            marks = [parse_mark(cell) for cell in row[2:]]
            if all(m is None for m in marks):
                yield "drop", "no valid marks"
```

```
                    continue
                yield "keep", (enrollment, name, marks)


def pass_one(path):
    with open(path, newline="", encoding="utf-8-sig") as fh:
        _, subjects = read_header(csv.reader(fh))
    stats = [SubjectStats() for _ in subjects]
    dropped = {}
    for kind, data in valid_rows(path):
        if kind == "drop":
            dropped[data] = dropped.get(data, 0) + 1
            continue
        for st, mark in zip(stats, data[2]):
            if mark is not None:
                st.add(mark)
    return subjects, stats, dropped


def pass_two(path, out_dir, subjects, stats):
    means = [round(st.mean, 2) for st in stats]
    grade_counts = {g: 0 for g, _ in GRADES}
    top = []                                    # min-heap of (avg, enrollment, name)
    kept = 0
    cleaned_path = os.path.join(out_dir, "cleaned_marks.csv")
    with open(cleaned_path, "w", newline="", encoding="utf-8") as out:
        writer = csv.writer(out)
        writer.writerow(["enrollment", "name", *subjects, "average", "grade", "filled_marks"])
        for kind, data in valid_rows(path):
            if kind == "drop":
                continue
            enrollment, name, marks = data
            filled = []
            for j, mark in enumerate(marks):
                if mark is None:
                    marks[j] = means[j]
                    stats[j].filled += 1
                    filled.append(subjects[j])
            avg = sum(marks) / len(marks)
            grade = grade_of(avg)
            grade_counts[grade] += 1
            kept += 1
            item = (round(avg, 2), _desc(enrollment), name, enrollment)
            if len(top) < TOP_K:
                heapq.heappush(top, item)
            elif item > top[0]:
                heapq.heapreplace(top, item)
            writer.writerow([enrollment, name, *(_fmt(m) for m in marks), f"{avg:.2f}",
                             grade, ";".join(filled)])
    top.sort(reverse=True)
    return kept, grade_counts, [(t[3], t[2], t[0]) for t in top]


def _desc(text):
    """Make smaller enrollment win ties inside a max-ordering heap."""
    return tuple(-ord(c) for c in text) + (1,)      # (1,) ends the key: "22" beats "221"


def _fmt(x):
    return str(int(x)) if float(x).is_integer() else f"{x:.2f}"


def write_summary(out_dir, subjects, stats):
    with open(os.path.join(out_dir, "summary.csv"), "w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["subject", "valid_marks", "filled_missing", "mean", "std_dev", "min",
                    "max", "pass_count", "pass_rate_percent"])
        for name, st in zip(subjects, stats):
            if st.count:
                w.writerow([name, st.count, st.filled, f"{st.mean:.2f}", f"{st.std:.2f}",
                            _fmt(st.low), _fmt(st.high), st.passed,
                            f"{100 * st.passed / st.count:.2f}"])
```

```
                else:
                    w.writerow([name, 0, st.filled, "", "", "", "", 0, ""])


#---------------------------------------------------------------------------------------------------------charts
def _axes(plt, title, figsize=(8, 4.6)):
    fig, ax = plt.subplots(figsize=figsize, dpi=150)
    fig.patch.set_facecolor("white")
    ax.set_title(title, loc="left", fontsize=13, color=INK, pad=12, fontweight="bold")
    for side in ("top", "right", "left"):
        ax.spines[side].set_visible(False)
    ax.spines["bottom"].set_color(GRID)
    ax.tick_params(colors=INK_2, labelsize=9, length=0)
    ax.set_axisbelow(True)
    return fig, ax


def save_charts(out_dir, subjects, stats, grade_counts, top):
    import matplotlib
    matplotlib.use("Agg")                       # no display needed
    import matplotlib.pyplot as plt

    files = []
    # 1. grade distribution
    fig, ax = _axes(plt, "Grade distribution")
    grades = [g for g, _ in GRADES]
    values = [grade_counts[g] for g in grades]
    bars = ax.bar(grades, values, color=BAR, width=0.6)
    ax.bar_label(bars, labels=[str(v) for v in values], padding=3, color=INK_2, fontsize=9)
    ax.set_ylabel("Students", color=INK_2)
    ax.yaxis.grid(True, color=GRID, linewidth=0.8)
    ax.set_xlabel("Grade (from average marks)", color=INK_2)
    files.append(_save(fig, plt, out_dir, "grade_distribution.png"))

    # 2. subject-wise average
    fig, ax = _axes(plt, "Subject-wise average marks")
    means = [st.mean for st in stats]
    bars = ax.bar(subjects, means, color=BAR, width=0.6)
    ax.bar_label(bars, labels=[f"{m:.1f}" for m in means], padding=3, color=INK_2, fontsize=9)
    ax.set_ylim(0, 100)
    ax.set_ylabel("Average marks (out of 100)", color=INK_2)
    ax.yaxis.grid(True, color=GRID, linewidth=0.8)
    if len(subjects) > 6:
        plt.setp(ax.get_xticklabels(), rotation=35, ha="right")
    files.append(_save(fig, plt, out_dir, "subject_average.png"))

    # 3. top performers
    fig, ax = _axes(plt, f"Top {len(top)} performers by average marks",
                    figsize=(8, 0.42 * max(len(top), 3) + 1.4))
    labels = [f"{name} ({enr})" for enr, name, _ in top][::-1]
    avgs = [avg for _, _, avg in top][::-1]
    bars = ax.barh(labels, avgs, color=BAR, height=0.6)
    ax.bar_label(bars, labels=[f"{a:.2f}" for a in avgs], padding=3, color=INK_2, fontsize=9)
    ax.set_xlim(0, 105)
    ax.set_xlabel("Average marks", color=INK_2)
    ax.xaxis.grid(True, color=GRID, linewidth=0.8)
    ax.spines["bottom"].set_visible(False)
    ax.spines["left"].set_visible(True)
    ax.spines["left"].set_color(GRID)
    files.append(_save(fig, plt, out_dir, "top_performers.png"))
    return files


def _save(fig, plt, out_dir, name):
    path = os.path.join(out_dir, name)
    fig.tight_layout()
    fig.savefig(path, facecolor="white")
    plt.close(fig)
    return path


#---------------------------------------------------------------------------------------------------------main
```

```python
def run(path, out_dir, out=print):
    os.makedirs(out_dir, exist_ok=True)
    subjects, stats, dropped = pass_one(path)
    if not any(st.count for st in stats):
        raise InputError("no valid student rows found")
    kept, grade_counts, top = pass_two(path, out_dir, subjects, stats)
    write_summary(out_dir, subjects, stats)
    charts = save_charts(out_dir, subjects, stats, grade_counts, top)
    out(f"Students kept: {kept}, dropped: {sum(dropped.values())}"
        + (f" ({', '.join(f'{v} {k}' for k, v in sorted(dropped.items()))})" if dropped else ""))
    out("Missing marks filled: " + ", ".join(f"{s}={st.filled}" for s, st in zip(subjects, stats)))
    for name in ["cleaned_marks.csv", "summary.csv"] + [os.path.basename(c) for c in charts]:
        out(name)


def main():
    if len(sys.argv) != 3:
        print("Usage: python ENROLLMENT_2_Q10.py marks.csv output_folder")
        sys.exit(1)
    try:
        run(sys.argv[1], sys.argv[2])
    except InputError as err:
        print(f"INPUT ERROR: {err}")
        sys.exit(1)
    except FileNotFoundError as err:
        print(f"ERROR: file not found: {err.filename}")
        sys.exit(1)
    except ImportError:
        print("ERROR: matplotlib is not installed. Run: pip install matplotlib")
        sys.exit(1)


if __name__ == "__main__":
    main()
```

Test cases — 8 of 8 passed

#	Test case	Input	Output (expected = actual)	Result

1	**Sample: marks.csv (17 rows, missing/invalid marks, duplicates)**
	sample
		`enrollment,name,Python,DBMS,OS`
		`2201,Asha,92,88,95`
		`2202,Bharat,78,,81`
		`2203,Chaitra,85,91,AB`
		`2204,Dev,45,52,38`
		`2205,Esha,67,73,70`
		`2206,Farhan,99,96,94`
		`2207,Gauri,35,28,41`
		`2208,Harsh,88,79,NA`
		`2209,Isha,,,`
		`2210,Jay,72,68,75`
		`2210,Jay Duplicate,10,10,10`
		`2211,Kavya,56,61,59`
		`,Nameless,80,80,80`
		`2212,Laksh,105,83,77`
		`2213,Meera,91,94,89`
		`2214,Nikhil,62,58,abc`
		`2215,Ojas,81,85,80`
			`Students kept: 14, dropped: 3 (1 duplicate enrollment, 1 missing enrollment, 1 no valid marks)`
			`Missing marks filled: Python=1, DBMS=1, OS=3`
			`cleaned_marks.csv`
			`summary.csv`
			`grade_distribution.png`
			`subject_average.png`
			`top_performers.png`
				PASS

2	**summary.csv contents**
	custom
		`(from the sample run)`	`subject,valid_marks,filled_missing,mean,std_dev,min,max,pass_count,pass_rate_percent`
			`Python,13,1,73.15,18.59,35,99,12,92.31`
			`DBMS,13,1,73.54,18.92,28,96,12,92.31`
			`OS,11,3,72.64,18.50,38,95,10,90.91`
				PASS

3	**Cleaned rows: blank, AB, 105 and abc replaced by subject mean**
	custom
		`2202 DBMS blank, 2203 OS 'AB', 2212 Python 105, 2214 OS 'abc'`	`2202,Bharat,78,73.54,81,77.51,A,DBMS`
			`2203,Chaitra,85,91,72.64,82.88,A+,OS`
			`2212,Laksh,73.15,83,77,77.72,A,Python`
			`2214,Nikhil,62,58,72.64,64.21,B+,OS`
				PASS

4	**Three chart images are valid PNG files**
	custom
		`file signatures`	`grade_distribution.png OK`
			`subject_average.png OK`
			`top_performers.png OK`
				PASS

#	Test case	Input	Output (expected = actual)	Result

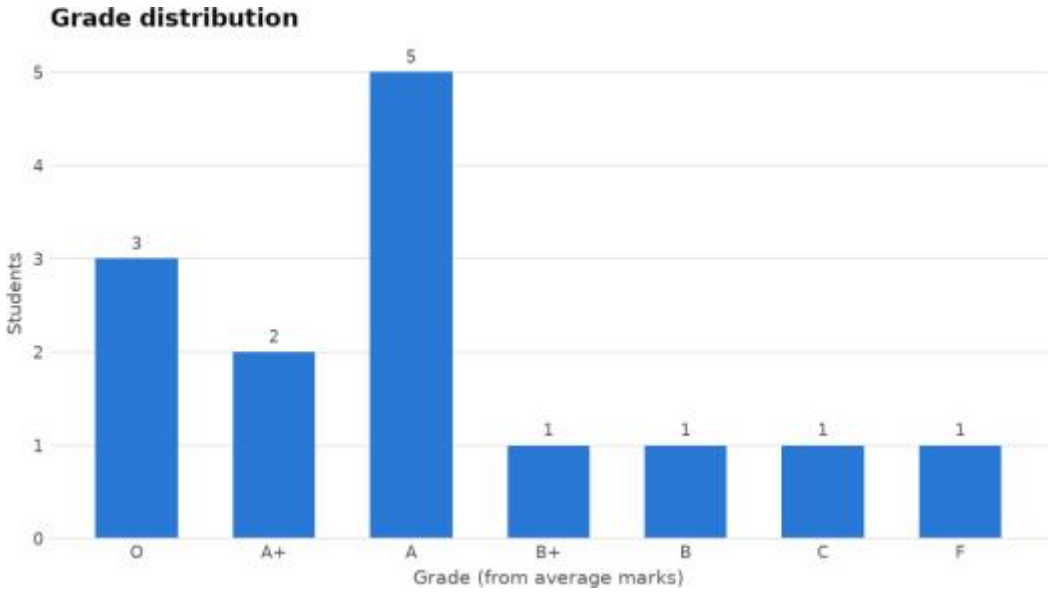
5	**Boundary: one student, one subject, mark exactly 40 (pass, grade C)**
	boundary
		`enrollment,name,Maths`
		`1,Solo,40`
			`Students kept: 1, dropped: 0`
			`Missing marks filled: Maths=0`
			`cleaned_marks.csv`
			`summary.csv`
			`grade_distribution.png`
			`subject_average.png`
			`top_performers.png`
			`- - -`
			`1,Solo,40,40.00,C,`
				PASS

6	**Invalid: wrong header** invalid	`roll,student,Maths`	`INPUT ERROR: header must be: enrollment,name,<subject>,...`	**PASS**

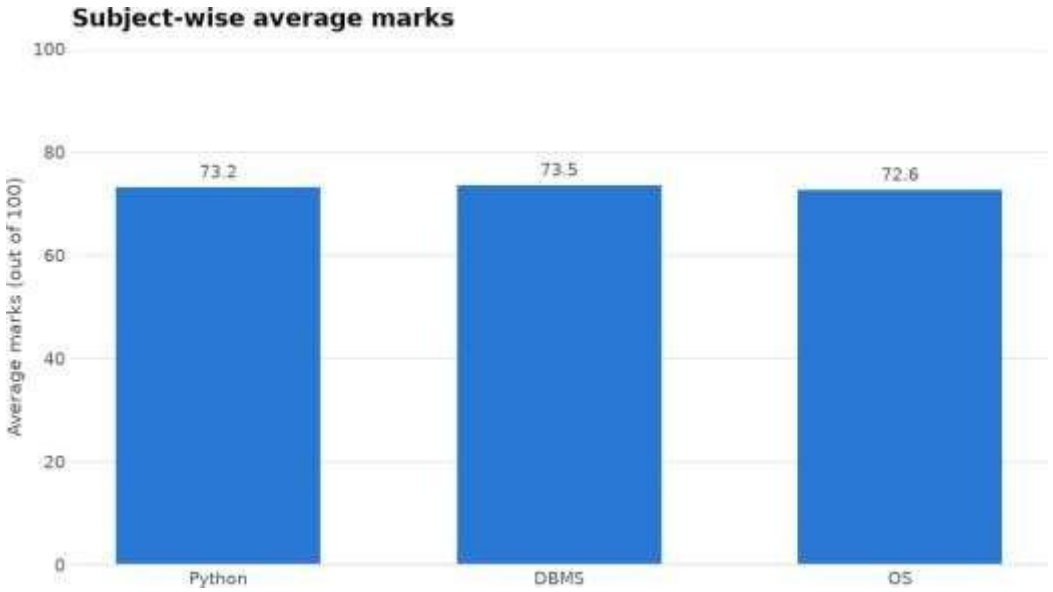
7	**Invalid: no usable rows**
	invalid
		`enrollment,name,Maths`
		`1,A,`
		`2,B,AB`
			`INPUT ERROR: no valid student rows found`	**PASS**

8	**Max size: 1,000,000 students x 10 subjects (2% missing)** performance	`random marks`	`finished in 24.71 s`	**PASS**

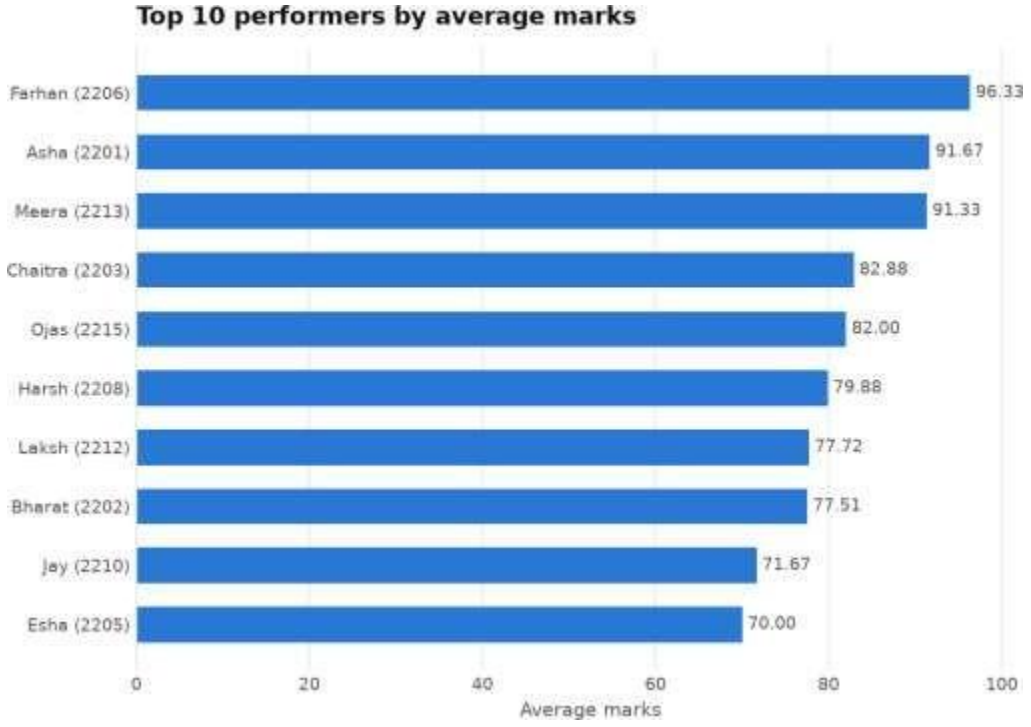
Charts produced from the sample marks.csv



grade_distribution.png



subject_average.png



top_performers.png

cleaned_marks.csv (sample run)

```
enrollment,name,Python,DBMS,OS,average,grade,filled_marks
2201,Asha,92,88,95,91.67,O,
2202,Bharat,78,73.54,81,77.51,A,DBMS
2203,Chaitra,85,91,72.64,82.88,A+,OS
2204,Dev,45,52,38,45.00,C,
2205,Esha,67,73,70,70.00,A,
2206,Farhan,99,96,94,96.33,O,
2207,Gauri,35,28,41,34.67,F,
2208,Harsh,88,79,72.64,79.88,A,OS
2210,Jay,72,68,75,71.67,A,
2211,Kavya,56,61,59,58.67,B,
2212,Laksh,73.15,83,77,77.72,A,Python
2213,Meera,91,94,89,91.33,O,
2214,Nikhil,62,58,72.64,64.21,B+,OS
2215,Ojas,81,85,80,82.00,A+,
```